



University
of Basel

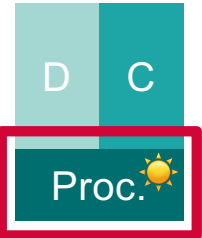
Chapter 5.2: Parallel Architectures and Programming Models – Memory Foundations of Distributed Systems

Fall Semester 2025

Prof. Dr. Florina Ciorba

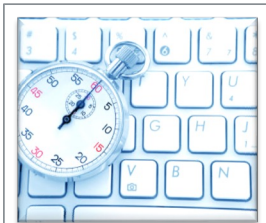


Addressed in The **Processing** Lecture Series



- Goals

Chapter 5.2:
Parallel architectures & models



Performance

```
00111101001010101
01001010100010101
01010ERROR011010
10100010100110001
01100010100100111
10100010100110001
```

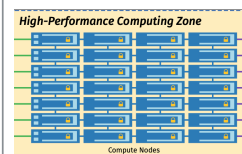
Resiliency



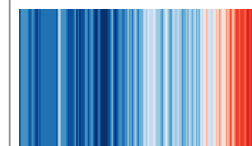
Extreme scalability



Energy



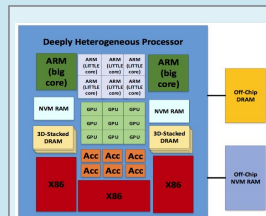
Security



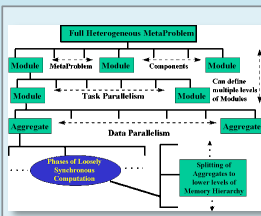
Sustainability



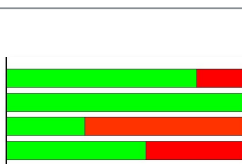
Convergence
HPC+Data+AI



Heterogeneity



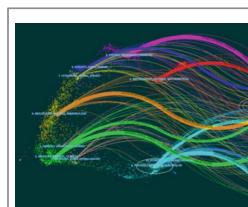
Multilevel parallelism



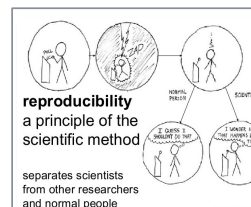
Load imbalance



Programming



Visualization



Reproducibility

Overview of Chapter 5.2: Parallel Architectures & Programming Models

a	Memory Architectures
b	Parallel Programming Models
c	Network Topologies

Learning objectives

Understand parallel computer architectures, including memory organization and network interconnections.

Understand their role in defining parallel programming models.

Parallel Computers and Their Programming

What We Learned So Far

A. Memory architectures

Machine model 1: Shared memory

⇒ Symmetric multiprocessing & UMA

Machine model 2: Distributed memory

⇒ Asymmetric multiprocessing & NUMA

Machine model 3: Hybrid distributed shared memory (combines 1+2)

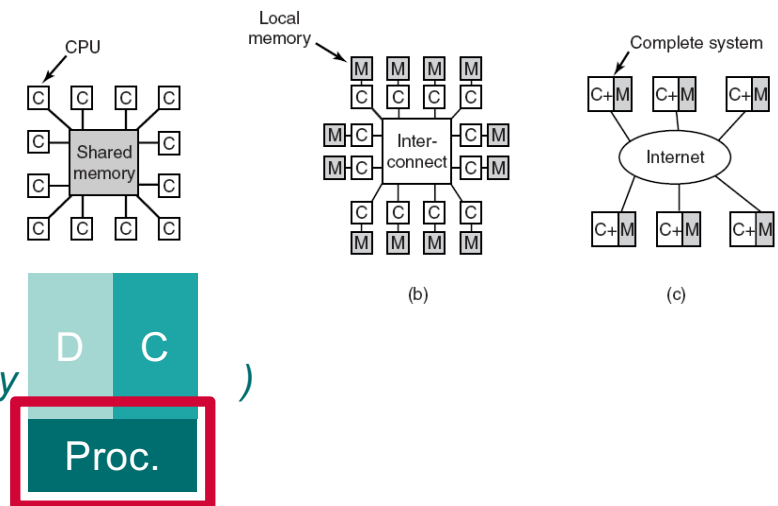
⇒ A/symmetric multiprocessing & NUMA, GAS

Machine model 4: Distributed shared memory (variation of 3)

⇒ A/symmetric multiprocessing & NUMA, DGAS

Machine model 5: Fully distributed systems (not parallel, not covered by)

What the machine offers
w.r.t. memory



B. Parallel programming models

C. Network topologies

Parallel Computers and Their Programming

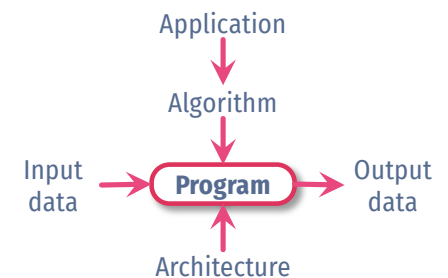
What We Learned So Far

A. Memory architectures

B. Parallel programming models (use explicit APIs)

1. Shared memory (**without** threads)
2. **Shared memory** (**with** threads)
3. **Distributed memory** / Message Passing
4. Data parallel
5. Hybrid **distributed shared** memory
6. Single Program Multiple Data (SPMD)
7. Multiple Program Multiple Data (MPMD)

What the programmer **sees**
w.r.t. memory



C. Network topologies

*“... these models are **not** specific to a particular type of machine or memory architecture. [...] any of these models can (theoretically) be implemented on any underlying hardware”*

[https://computing.llnl.gov/tutorials/parallel_comp]

Parallel Computers and Their Programming

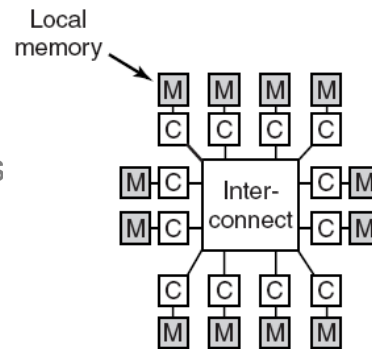
As Easy as A, B, C

A. Memory architectures

B. Parallel programming models

C. Network topologies

1. On chip networks
2. System/Storage area networks
3. Local area networks
4. Wide area networks



What the machine **offers**
w.r.t. communication

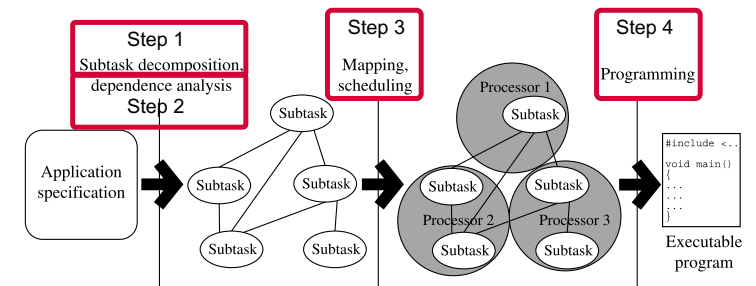


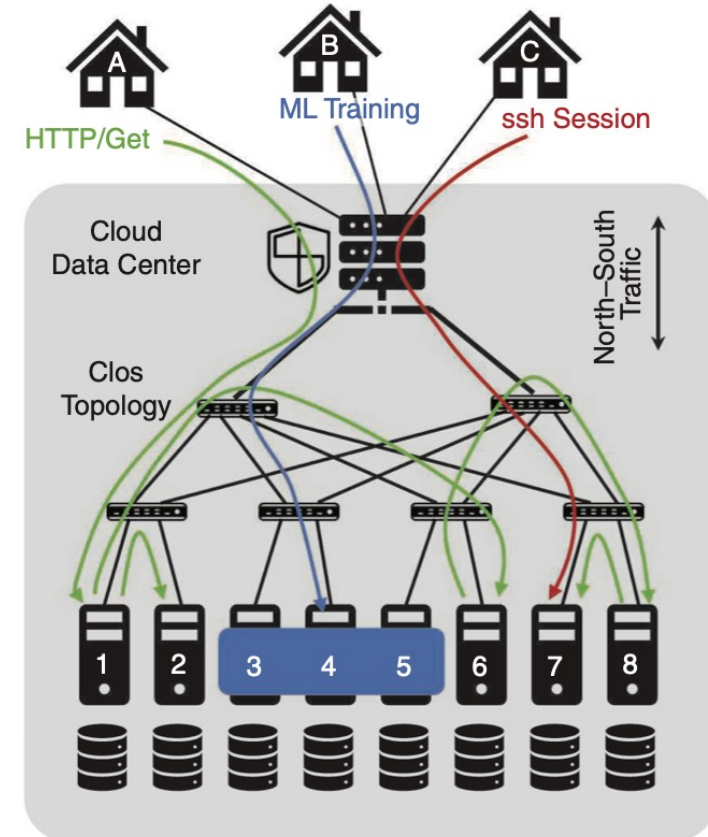
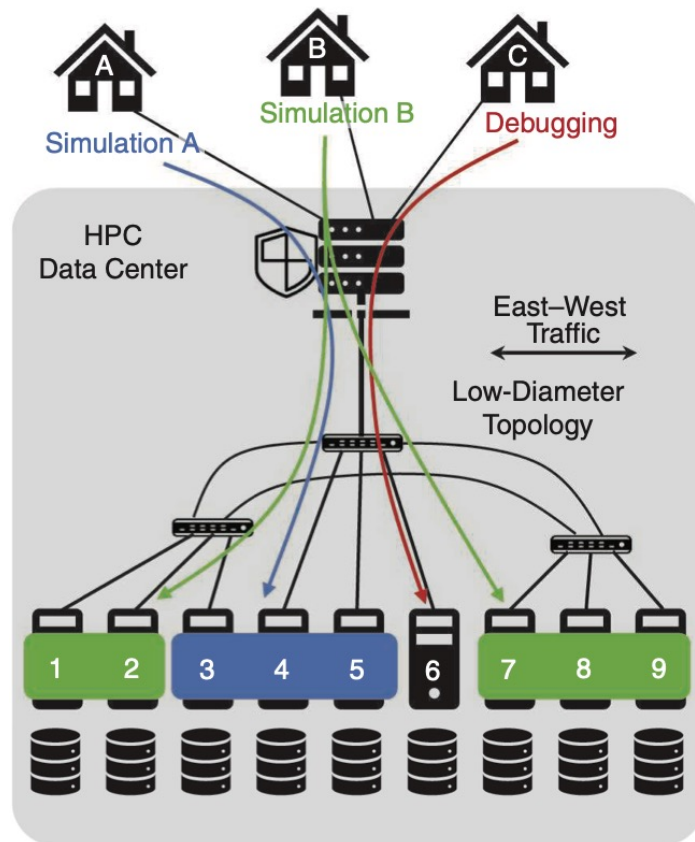
Figure 1.1. Parallel programming—process of parallelization.

*“The **interconnection network** is the **heart** of parallel architecture.”*

[Chuan-Lin and Tse-Yun Feng, Interconnection Networks for Parallel and Distributed Processing (1984)]

Motivating Example 1

Interconnection Network in HPC and Cloud Data Centers

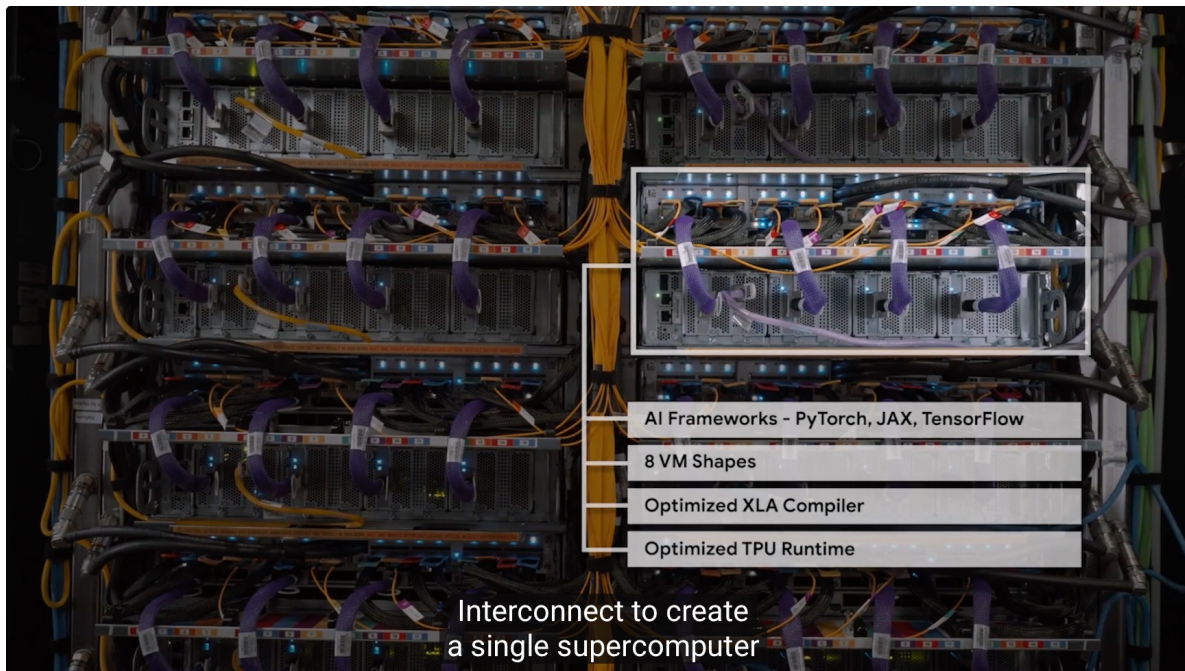


HPC data center (on prem) serves three **distributed** simulation workloads where clients are **not waiting** for an immediate answer

Cloud data center serves multiple customers with different **interactive** services, some **distributed** (ML training job on machines 3–5)

Motivating Example 2

Photos and Search on Google Cloud TPU Data Center



https://youtu.be/FsxthdQ_sL4 (2'39")

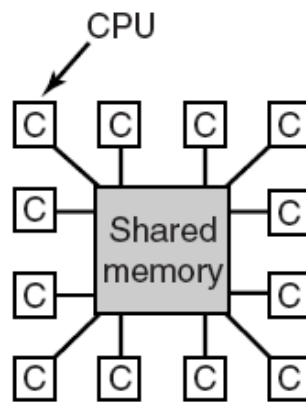
<https://cloud.google.com/tpu>

4096 v4 TPUs
3D Torus Network
> 1 Exaflop/s (bfloat16)

Google Cloud TPU
NVLink
ICI
DCN

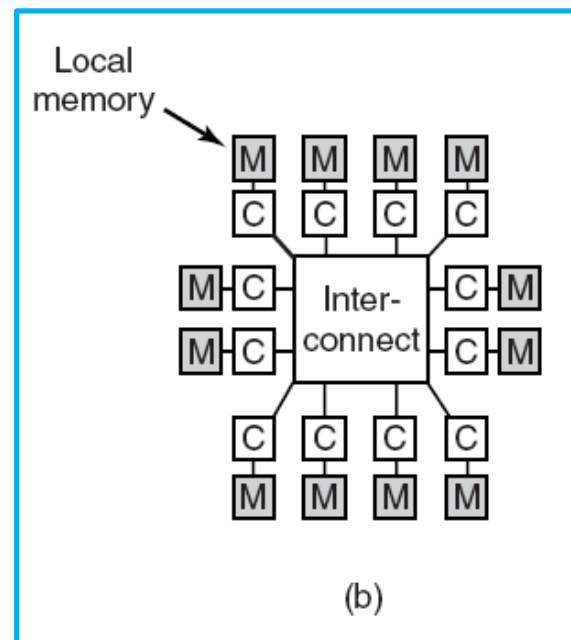
- System Architecture of these ASICs <https://cloud.google.com/tpu/docs/system-architecture-tpu-vm>
- NVIDIA NVLink technology across GPUs (within and between servers)
- Inter-Chip Interconnect within a single TPU pod (**2D Torus v5**, **3D Mesh/Torus v4**, **2D torus v3**)
- Data Center Network (LAN) across multiple pods

Recall Multiple Processor Computer Systems?



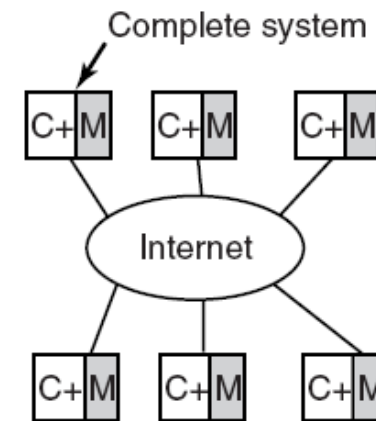
(a)

Accessing a memory word:
2-10 nsec



(b)

Sending/receiving short messages:
10-50 usec



(c)

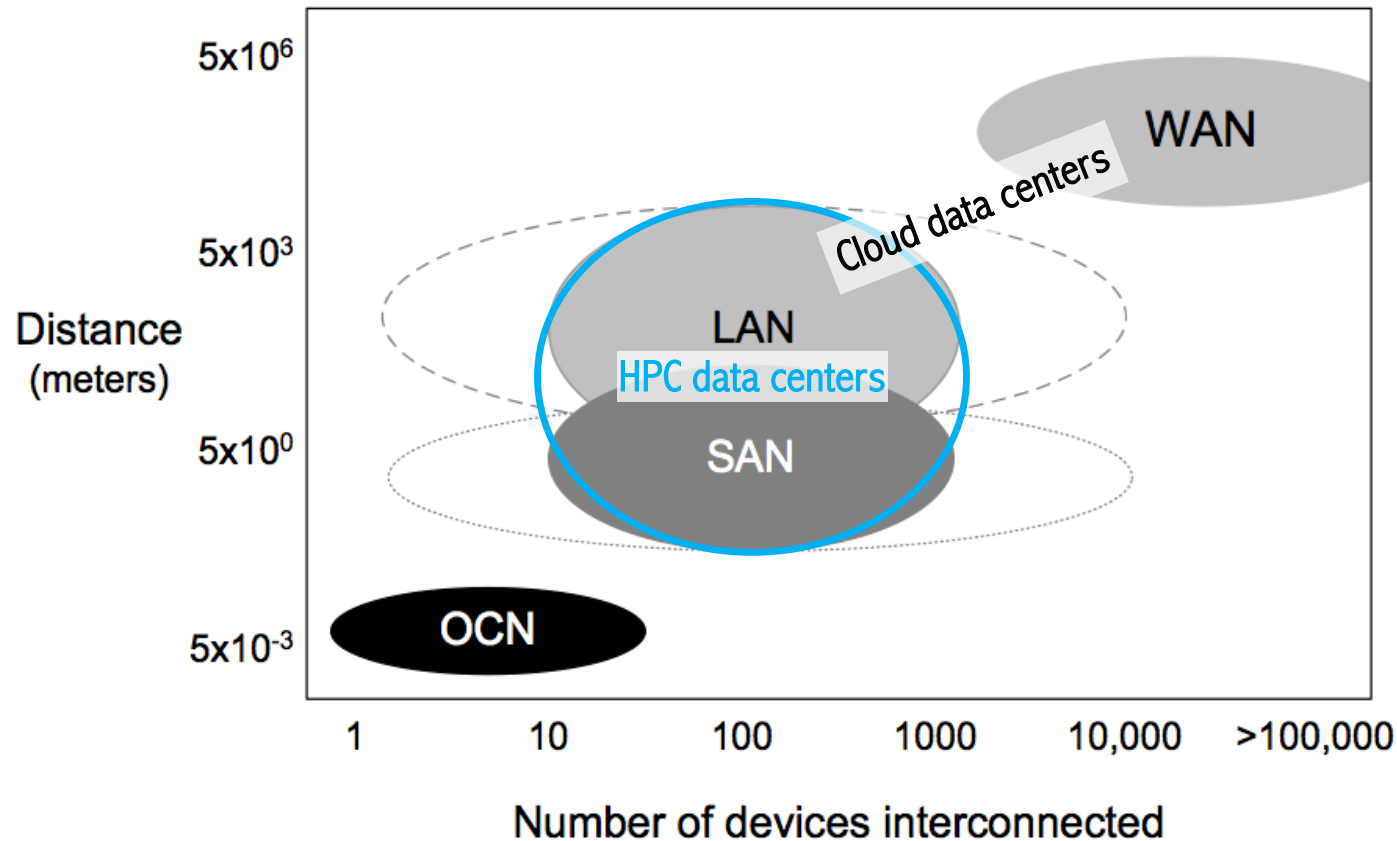
Sending/receiving short messages:
10-100 msec

(a) A shared-memory multiprocessor: common memory + full access

(b) A **parallel distributed-memory** (message-passing) **multicomputer**: not common memory + no full access, with a LAN

(c) A wide area distributed multicomputer system which typically consists of shared- (locally) and/or distributed-memory (globally) units

Types of Fixed Networks



Type 1: OCN — On Chip Network

Type 2: SAN — System/Storage Area Network (all nodes have access to same storage through the network, e.g., miniHPC)

Type 3: LAN — Local Area Network (more loosely coupled than SAN)

Type 4: WAN — Wide Area Network (long haul, require internet)

[Source: Hennessy/Patterson]

Fixed Network Type 3: Local Area Networks (LAN)

Generic Cluster Computer Architecture

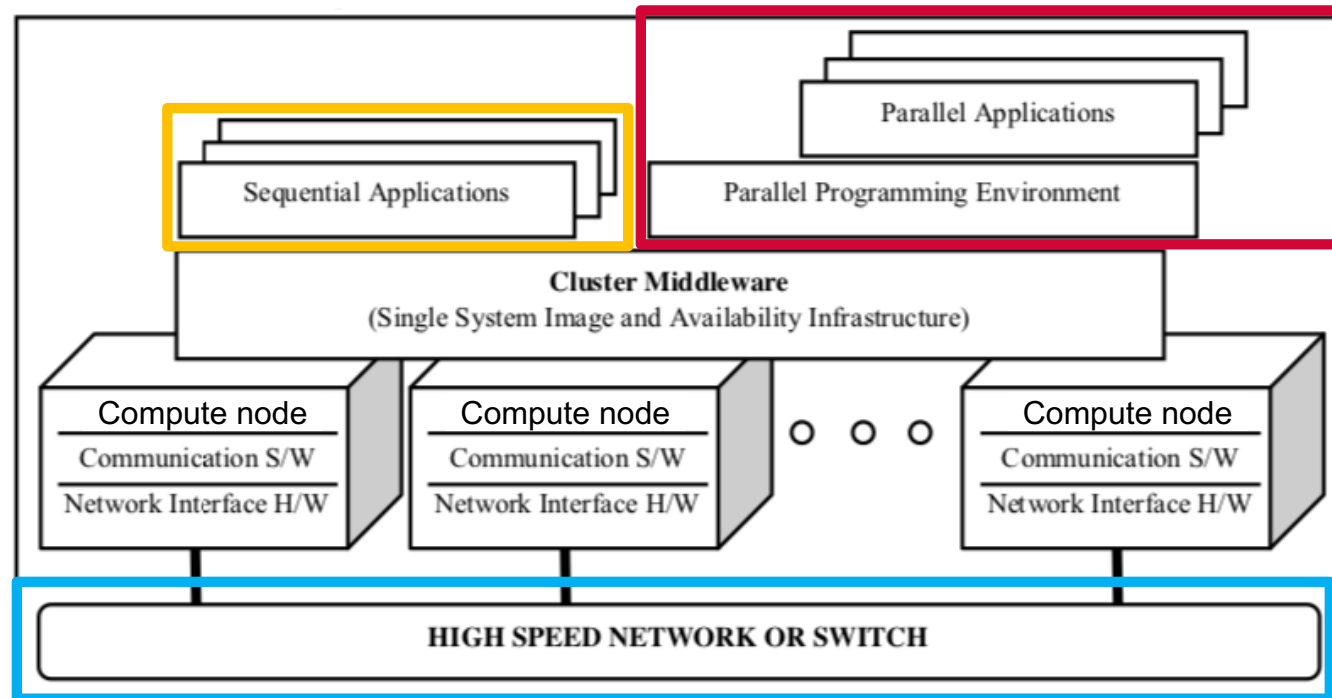


Figure 4.1: Cluster Computer Architecture

[Source: <https://arxiv.org/pdf/1311.3070.pdf>]

Fixed Network Type 3: Local Area Networks (LAN)

Static Vs. Dynamic Interconnection Networks

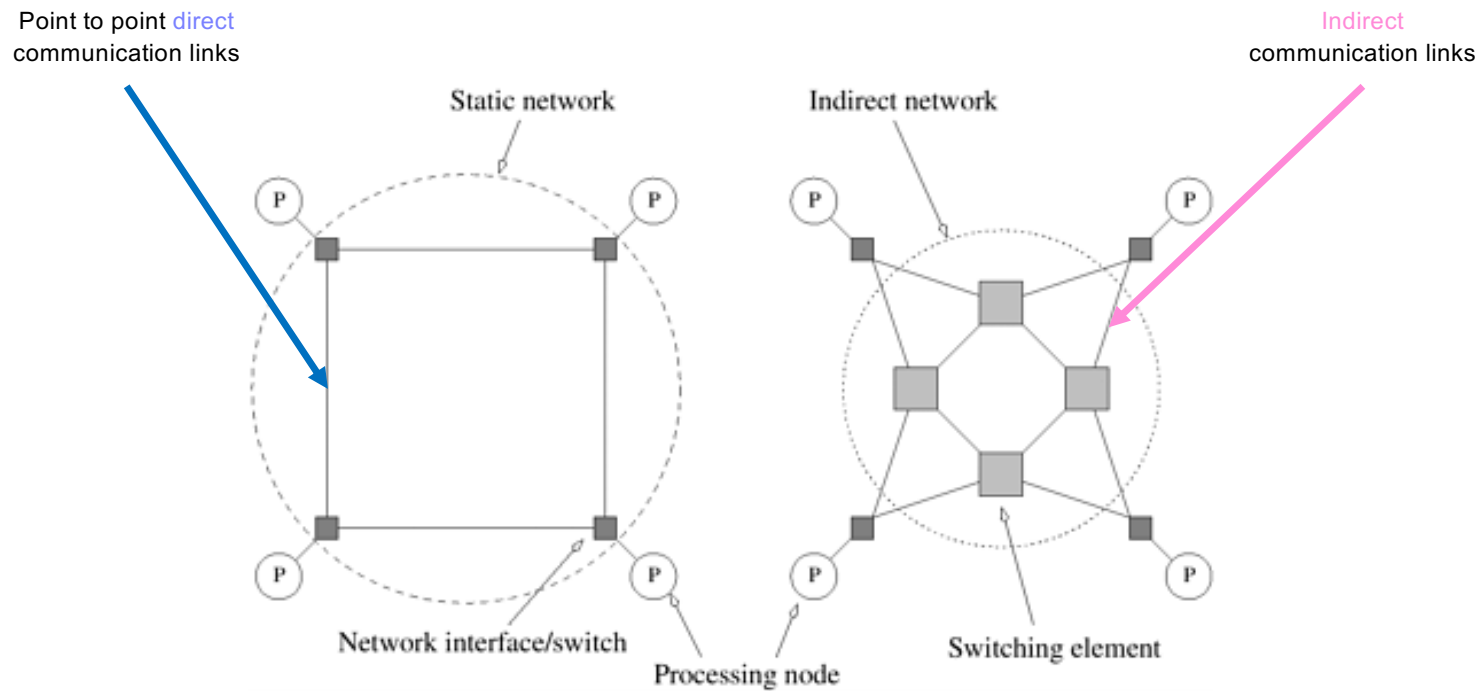


Figure 2.6. Classification of interconnection networks:

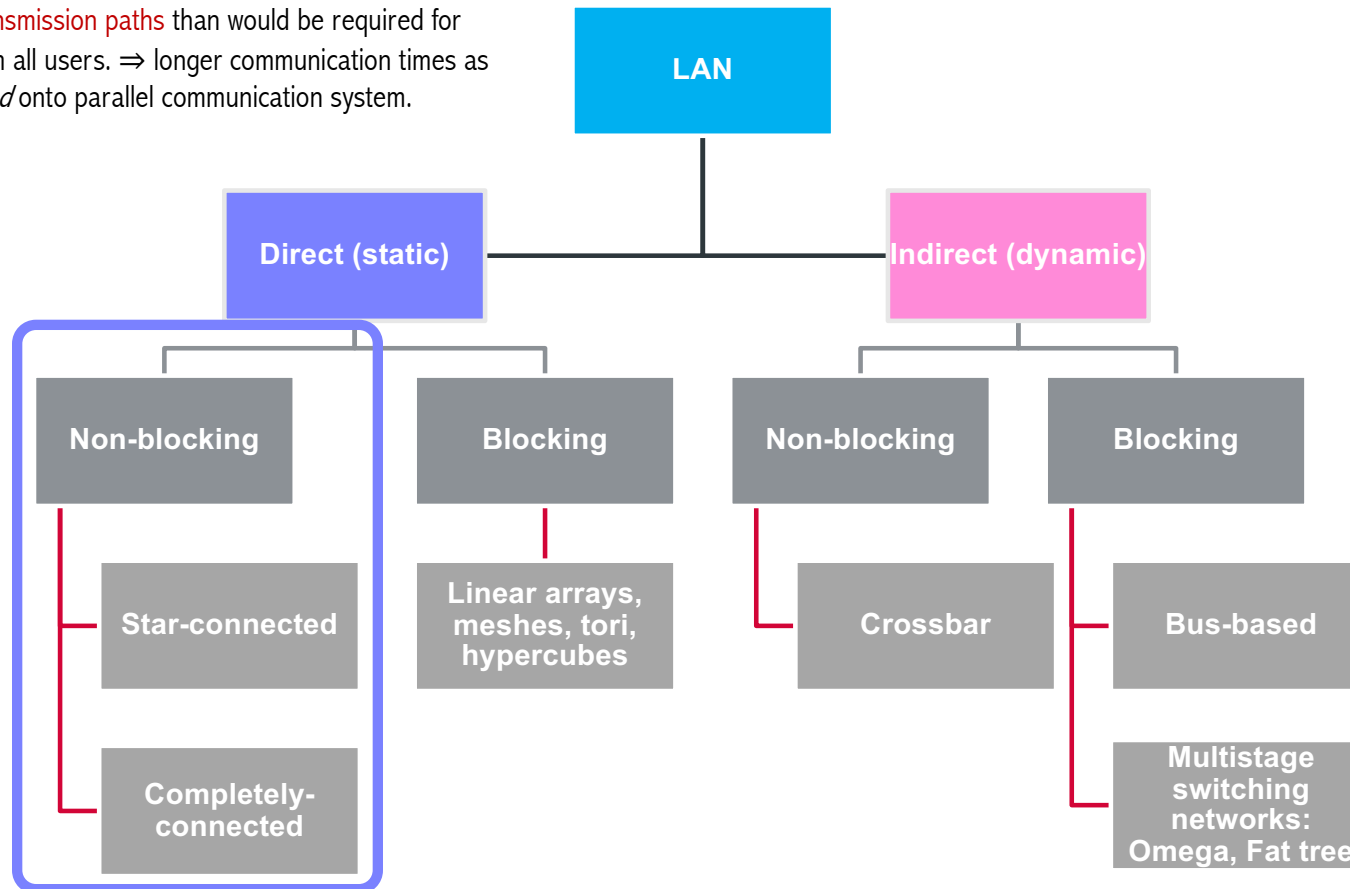
(a) a static (direct) network (2x2 mesh);

(b) a dynamic (indirect) network

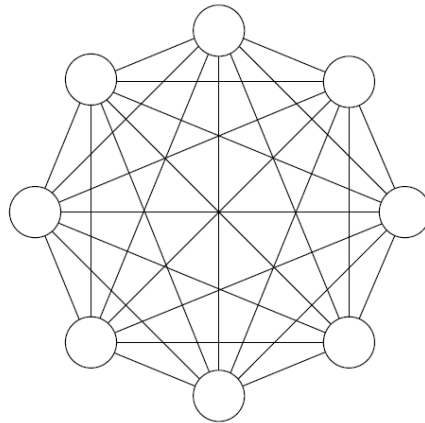
Fixed Network Type 3: Local Area Networks (LAN)

LAN Types

Blocking: network has *fewer transmission paths* than would be required for simultaneous transmissions from all users. $\Rightarrow$ longer communication times as concurrent communication *folded* onto parallel communication system.



LAN > Direct (Static) > Non-Blocking: Star- and Completely-Connected Networks

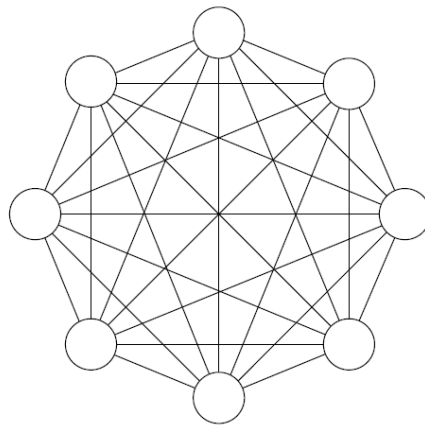


(a)

Completely-connected
(8 nodes, 64 links)

++ Non-blocking
-- p^2 links

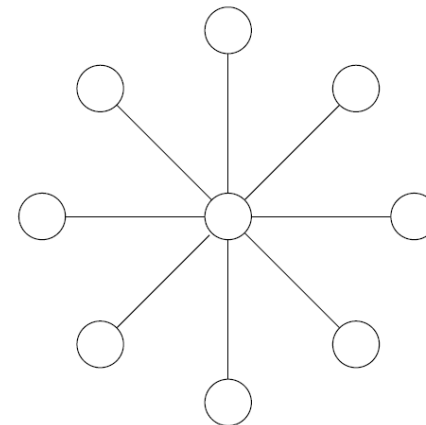
LAN > **Direct (Static)** > ~**Non-Blocking**: Star- and Completely-Connected Networks



(a)

Completely-connected
(8 nodes, 64 links)

++ Non-blocking
-- p^2 links



(b)

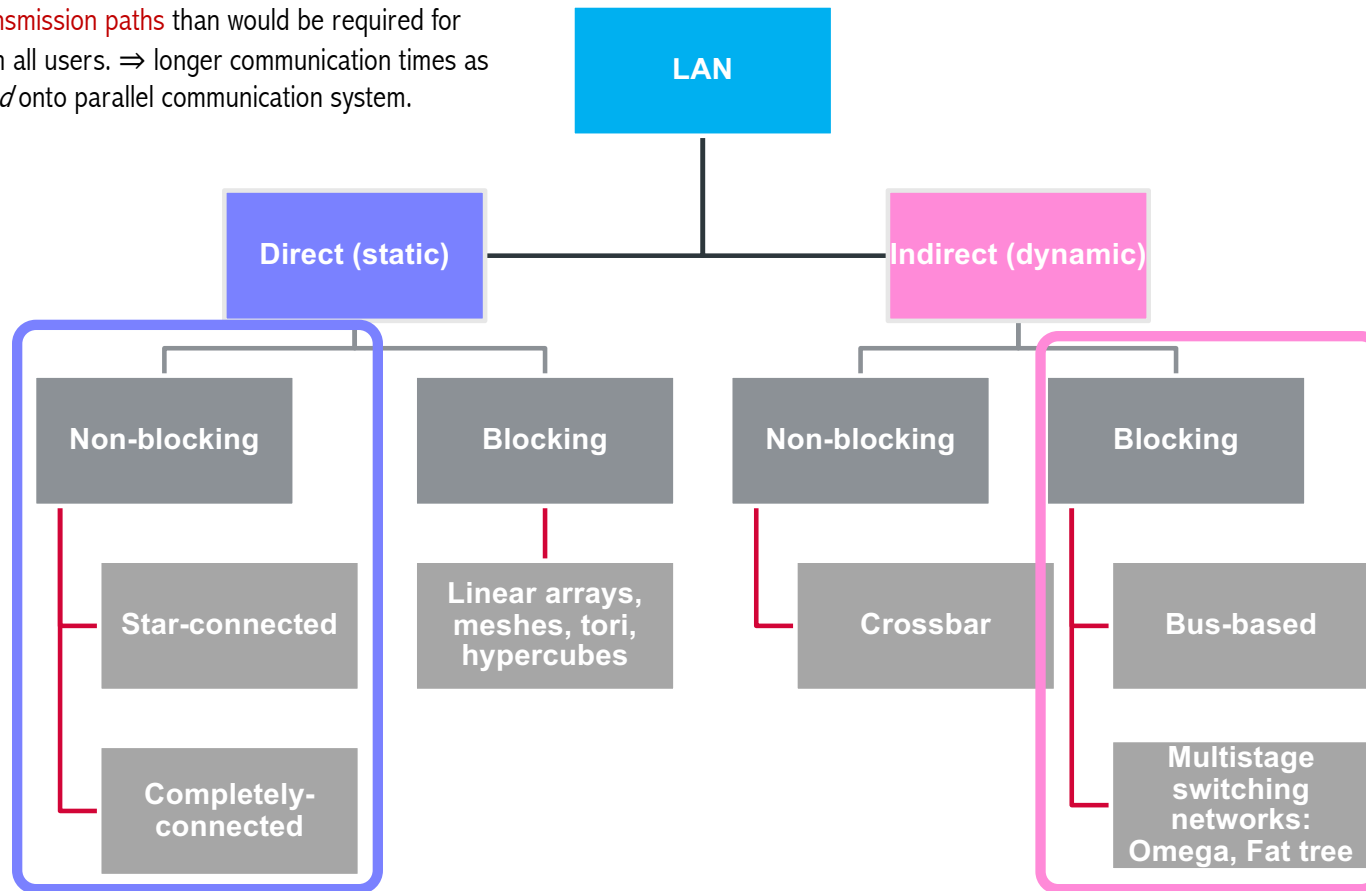
Star-connected
(9 nodes, 8 links)

-- ~ Non-blocking
++ ~ p links

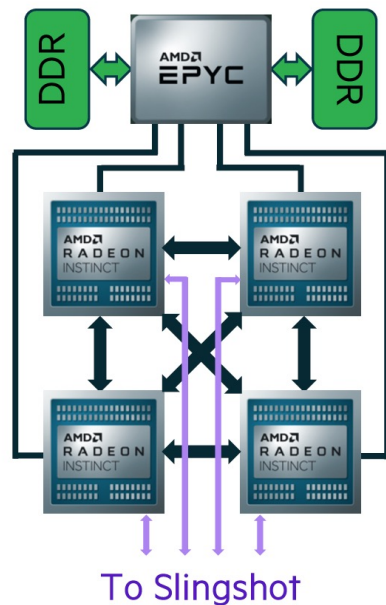
Fixed Network Type 3: Local Area Networks (LAN)

LAN Types

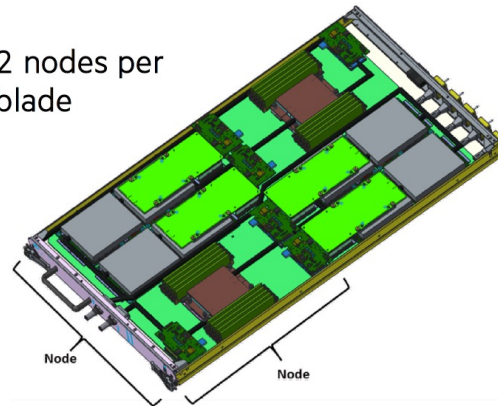
Blocking: network has *fewer transmission paths* than would be required for simultaneous transmissions from all users. $\Rightarrow$ longer communication times as concurrent communication *folded* onto parallel communication system.



LAN > Direct (Static) > ~Non-Blocking: Completely-Connected Network Example: Frontier On-Chip Network (Top 2)

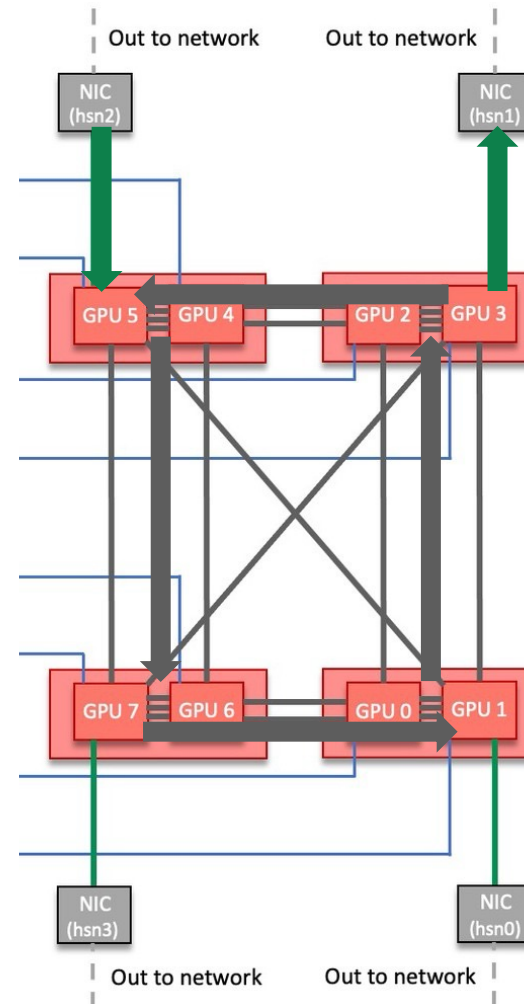


2 nodes per blade



COPYRIGHT 2020 HPE

AMD GPU
(ORNL)



Network = 200 Gbps
Intranode = 400 Gbps
(x2 or x4)

Left: <https://www.olcf.ornl.gov/frontier/>

Right: https://docs.olcf.ornl.gov/systems/frontier_user_guide.html#connecting

LAN > Indirect (Dynamic) > ~Non/Blocking: Completely-Connected Multistage Interconnection Network Example: Frontier Cross-Node Network (Top 2)



Switches connected
all-to-all in a group

Interconnect
HPE Slingshot

Node Injection
4x 25 GB/s

Interface
Libfabric/OFI

Topology
Dragonfly

Groups
connected
all-to-all

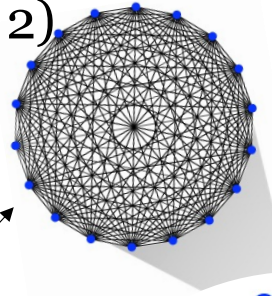
90+ miles of cables



LAN > Indirect (Dynamic) > ~Non/Blocking:
Completely-Connected Multistage Interconnection Network
Example: Frontier Dragonfly Network (Top 2)



Switches connected
all-to-all in a group



⇒ Many supercomputers use Dragonfly
Fugaku (Top 7), ALPS (Top 8), LUMI (Top 9)

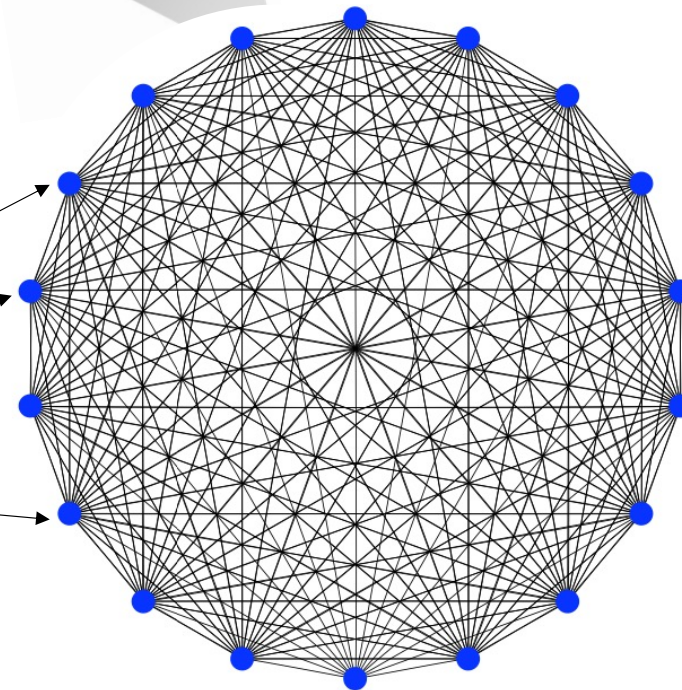
Interconnect
HPE Slingshot

Node Injection
4x 25 GB/s

Interface
Libfabric/OFI

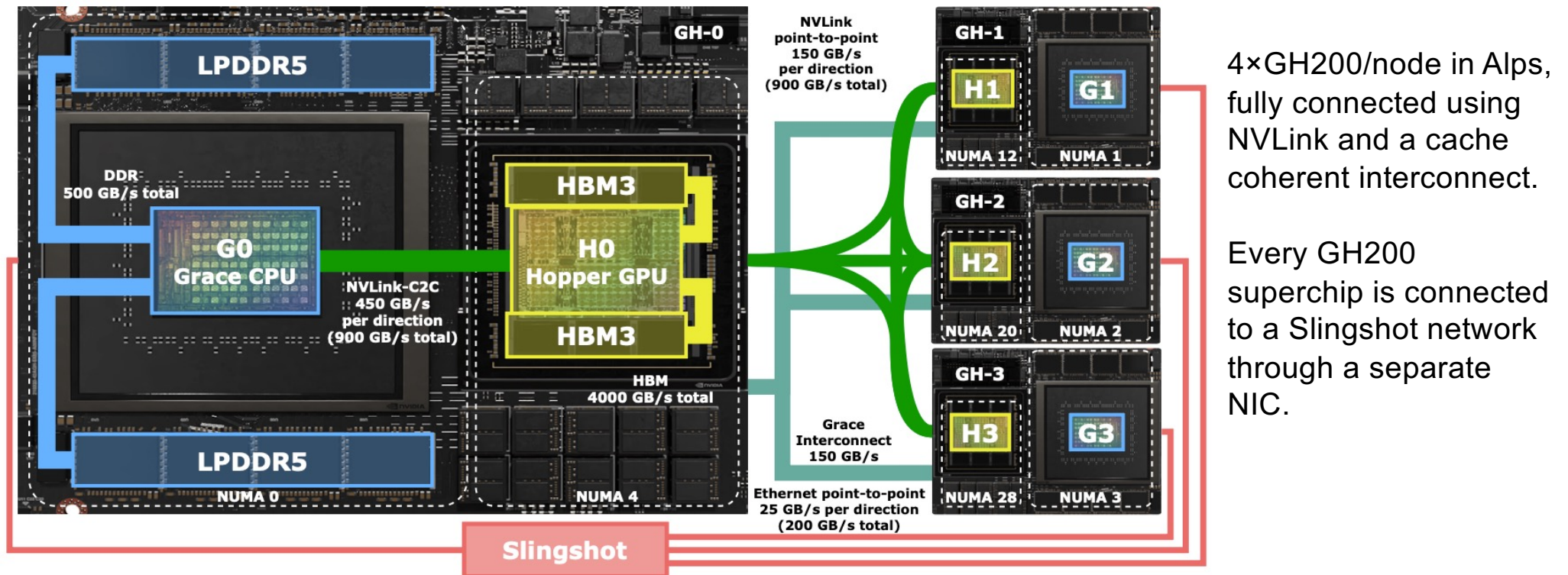
Topology
Dragonfly

Groups
connected
all-to-all



90+ miles of cables

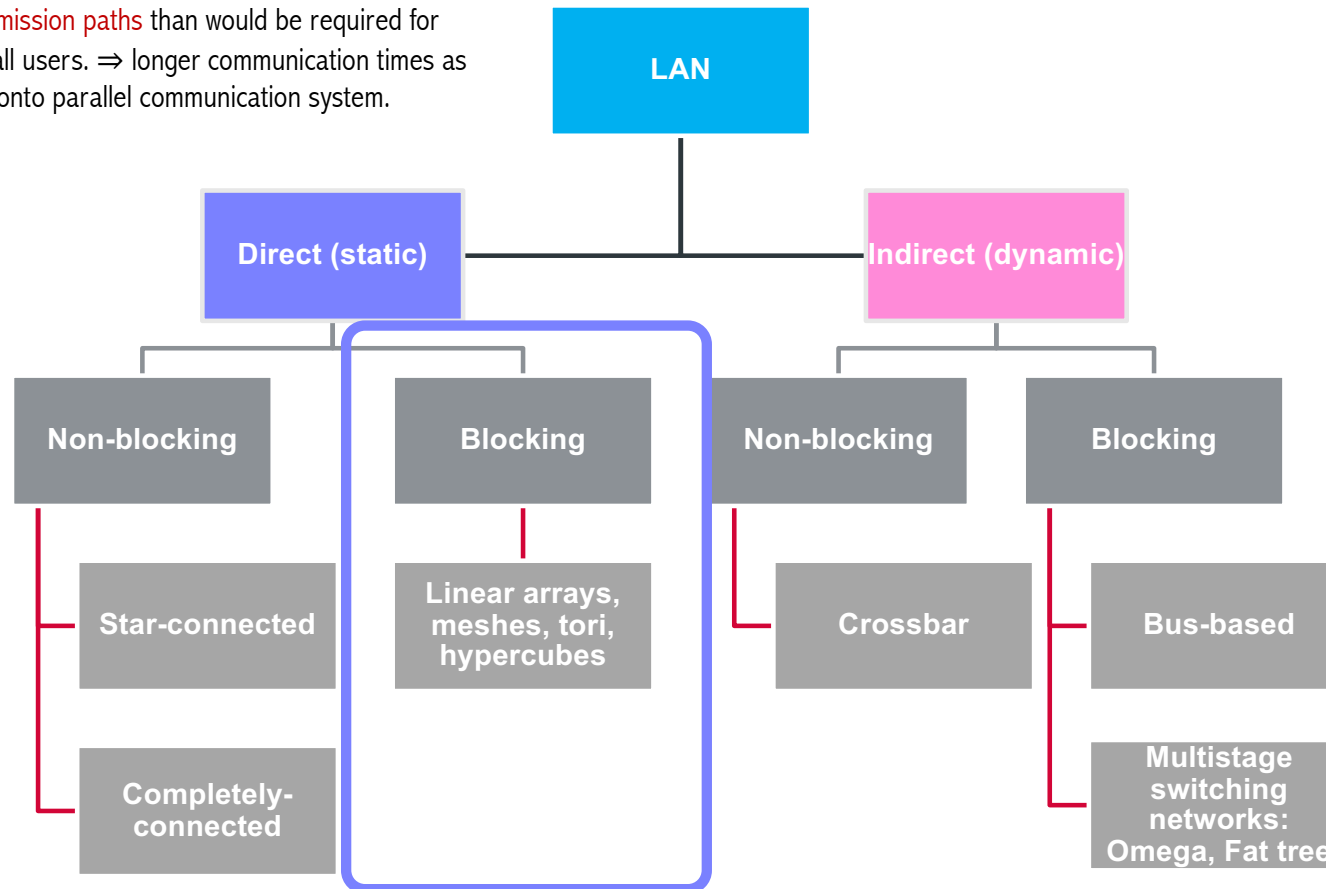
🔗 LAN > Indirect (Dynamic) > Blocking:
Bus-Based Interconnection Network
Example: Alps Node-Level Network (Top 8)



Fixed Network Type 3: Local Area Networks (LAN)

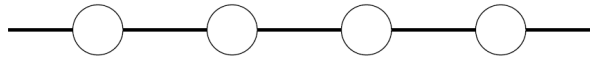
LAN Types

Blocking: network has *fewer transmission paths* than would be required for simultaneous transmissions from all users. $\Rightarrow$ longer communication times as concurrent communication *folded* onto parallel communication system.

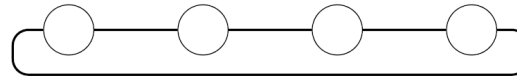


LAN > Direct (Static) > Blocking: k-d Meshes: Linear Arrays, Meshes, Tori

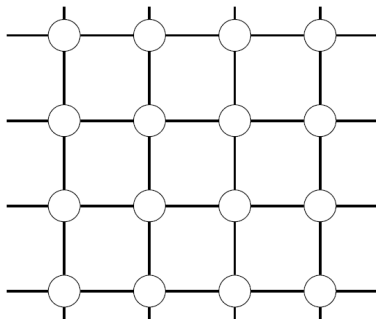
⇒ Many systems use d-Torus
Fugaku (Top 7), Google Cloud TPU



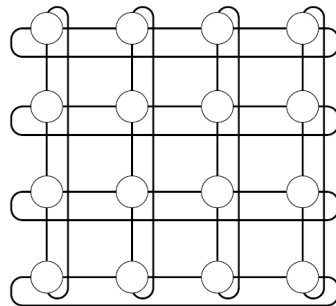
Linear Array without wrap-around 4-ary 1-cube



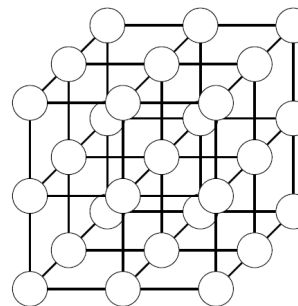
Linear Array **with** wrap-around (ring) 4-ary 1-cube



4-ary 2-cube
2-D Mesh (grid)



4-ary 2-cube **with** wrap-around
2-D **Torus**



3-ary 3-cube
3-D Mesh (cube)

Each dimension has $k = \sqrt[d]{p}$ nodes with a node identified by a d-tuple (i) , (i, j) , or (i, j, k)

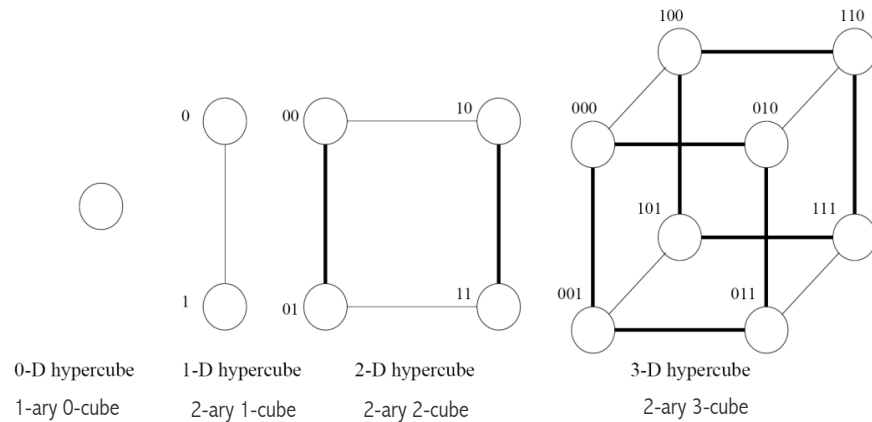
...

3-ary 3-cube **with** wrap-around
3-D **Torus**

k-d meshes (**k-ary d-cube**) refers to the class of topologies consisting of **d dimensions**, **k nodes** per dimension, and **$p = k^d$ processors** (e.g., 4-ary 1-cube, 4-ary 2-cube, 3-ary 3-cube)

LAN > Direct (Static) > Blocking:

k-d Meshes: Hypercube or D-Cube Topology



LAN > Direct (Static) > Blocking:

k-d Meshes: Hypercube or D-Cube Topology

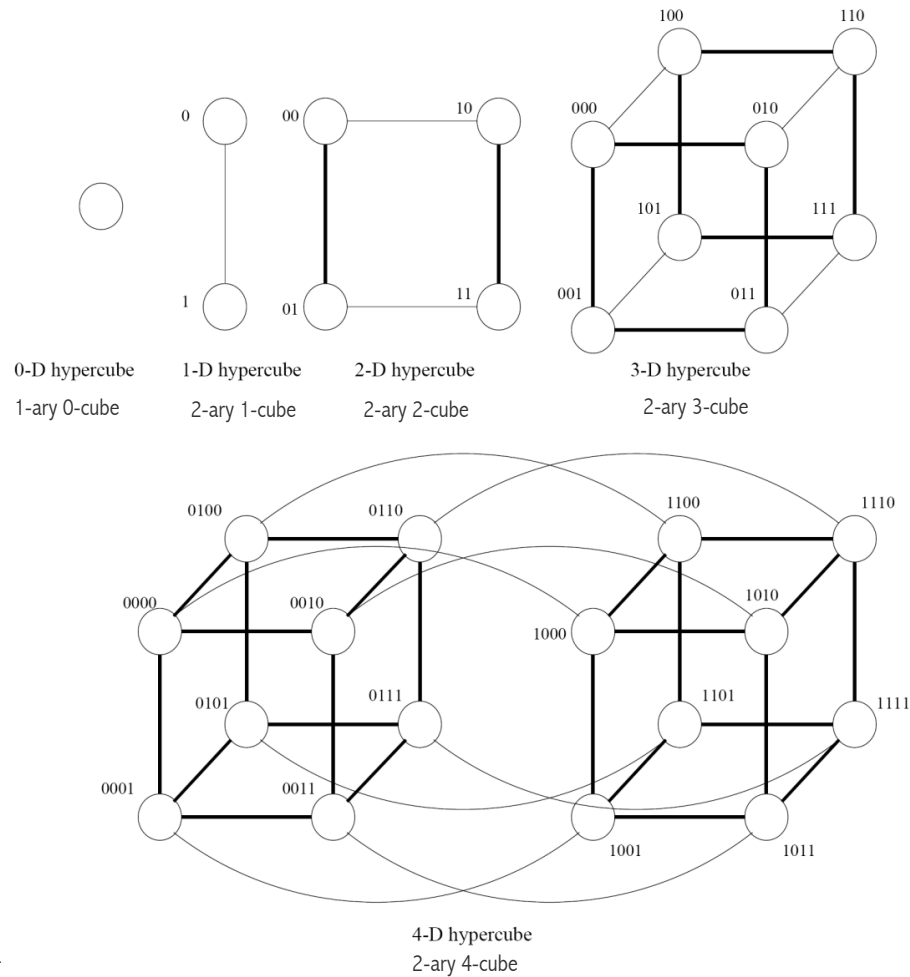
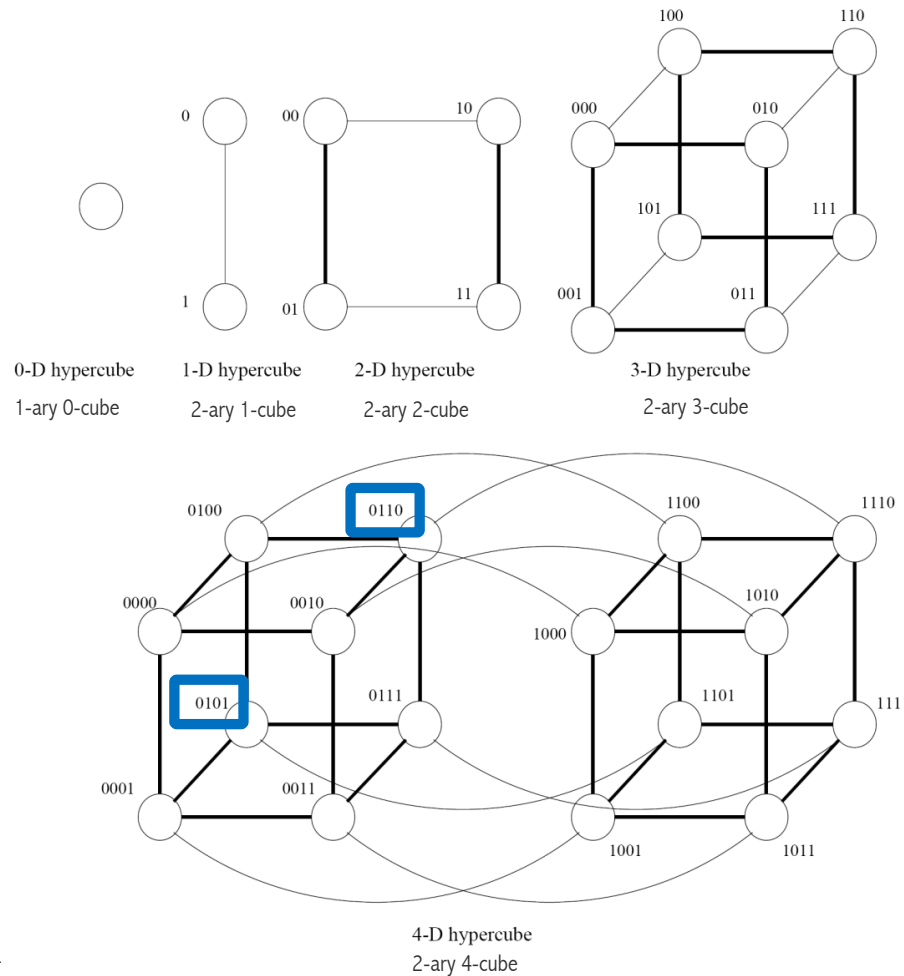


Figure 2.17 Construction of hypercubes from hypercubes of lower dimension.

LAN > Direct (Static) > Blocking: k-d Meshes: Hypercube or D-Cube Topology

⇒ Many systems use hypercubes
Fugaku (Top 7), Jugene (decommissioned)



Particular case of k-d meshes

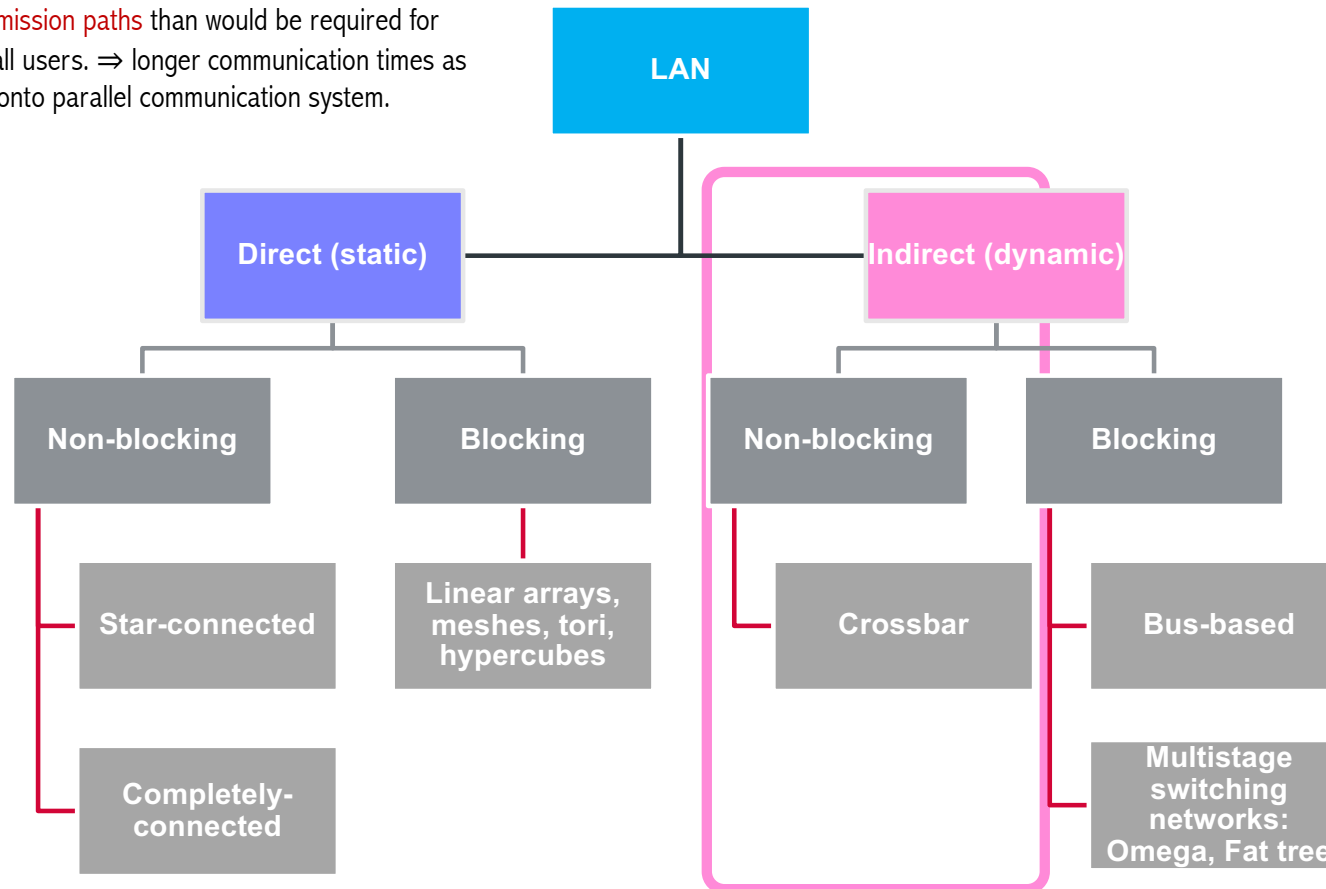
- Only $k=2$ nodes (2-ary) interconnected along each dimension
- **Requires:** d dimensions (d-cube) large enough to interconnect all p nodes in the system
 - $p = 2^d$ nodes
 - $d = \log_2 p$ dimensions
 - $(p \log p)/2$ links
- ++ Linearly increasing diameter $\log_2 p$
- 4-D hypercube numbering scheme: prefix labels for one sub-cube with “0” and the other with “1”
- Distance between 2 nodes = #different bits in the two labels
 - 0110 and 0101 are 2 links apart

Figure 2.17 Construction of hypercubes from hypercubes of lower dimension.

Fixed Network Type 3: Local Area Networks (LAN)

LAN Types

Blocking: network has *fewer transmission paths* than would be required for simultaneous transmissions from all users. $\Rightarrow$ longer communication times as concurrent communication *folded* onto parallel communication system.



LAN > Indirect (Dynamic) > Non-Blocking: UMA Multiprocessors Using Crossbar Switches

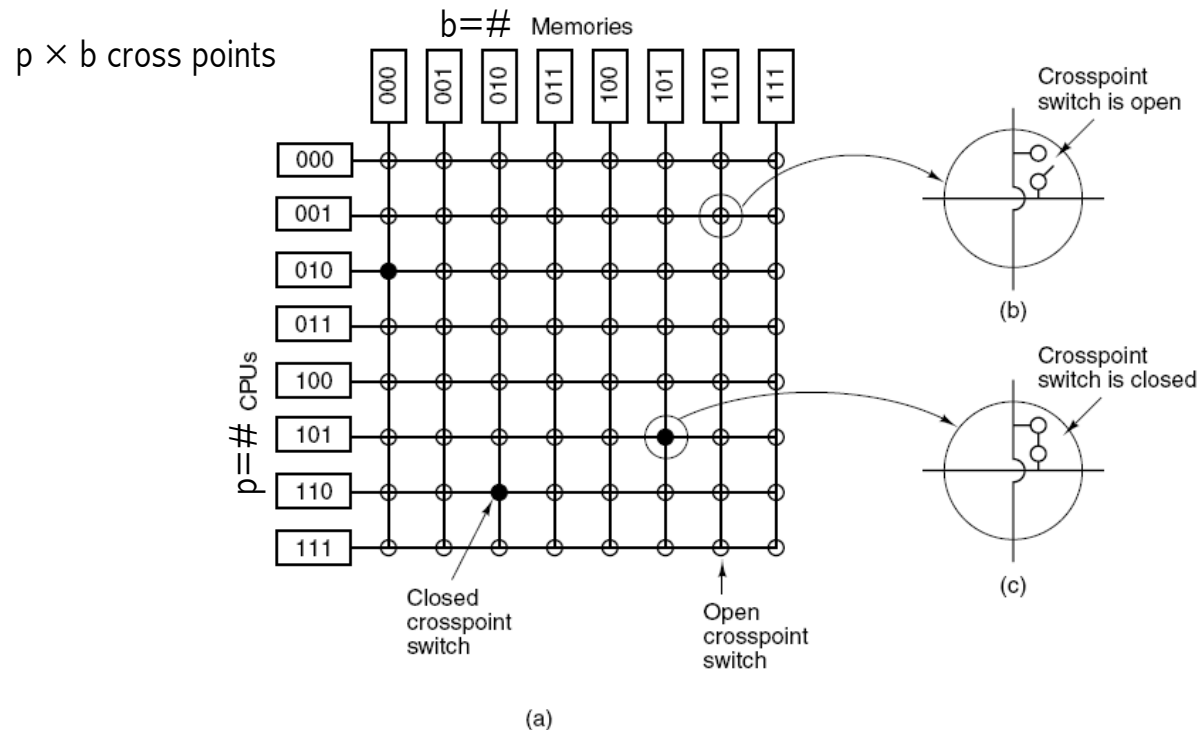


Figure 8-3.

- (a) An 8×8 crossbar switch
- (b) An open cross point (no connection)
- (c) A closed cross point (connection)

Requires: $p \times b$ switching elements (p in, b out) $\Rightarrow p > b \rightarrow \Omega(p^2)$
 ++ High bandwidth (bit/s)
 ++ **Non-blocking**
 -- **Poor cost scalability** ($p=100 \times b=100 = 10k$ cross points!!!)

LAN > Indirect (Dynamic) > Non-Blocking: UMA Multiprocessors Using Crossbar Switches: Example ROSETTA Switch

$b=8$ columns of tiles

$4 \times 8 = 32$ tiles

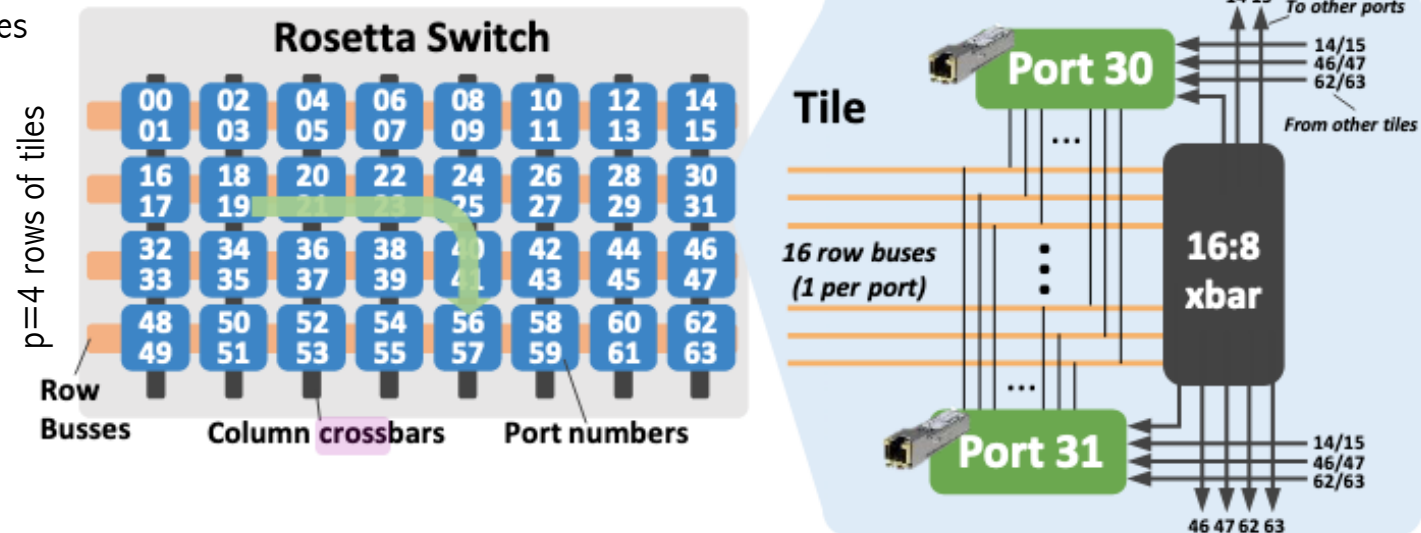


Fig. 1: ROSETTA switch tiled structure.

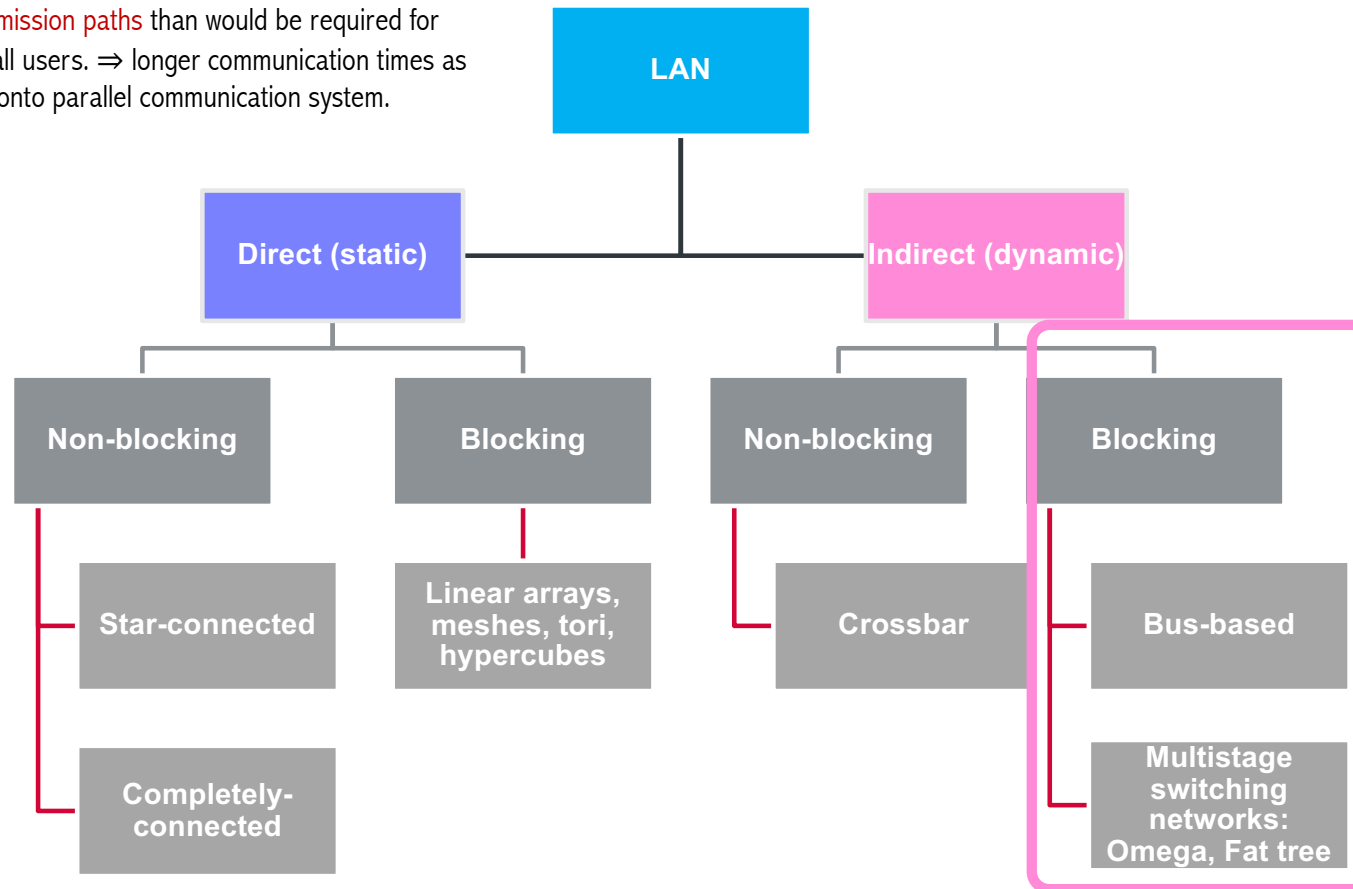
ROSETTA Switch == core of the Slingshot interconnect, forming a Dragonfly topology (switches connect all-to-all in a group)
Slingshot (by HPE/Cray) powers up the top supercomputers

- 64 ports per switch, at 200 Gb/s per direction
- 32 tiles implement a crossbar between the 64 ports, 2 switch ports / tile
- **!** ROSETTA has a mean and median switch latency of 350 nanoseconds

Fixed Network Type 3: Local Area Networks (LAN)

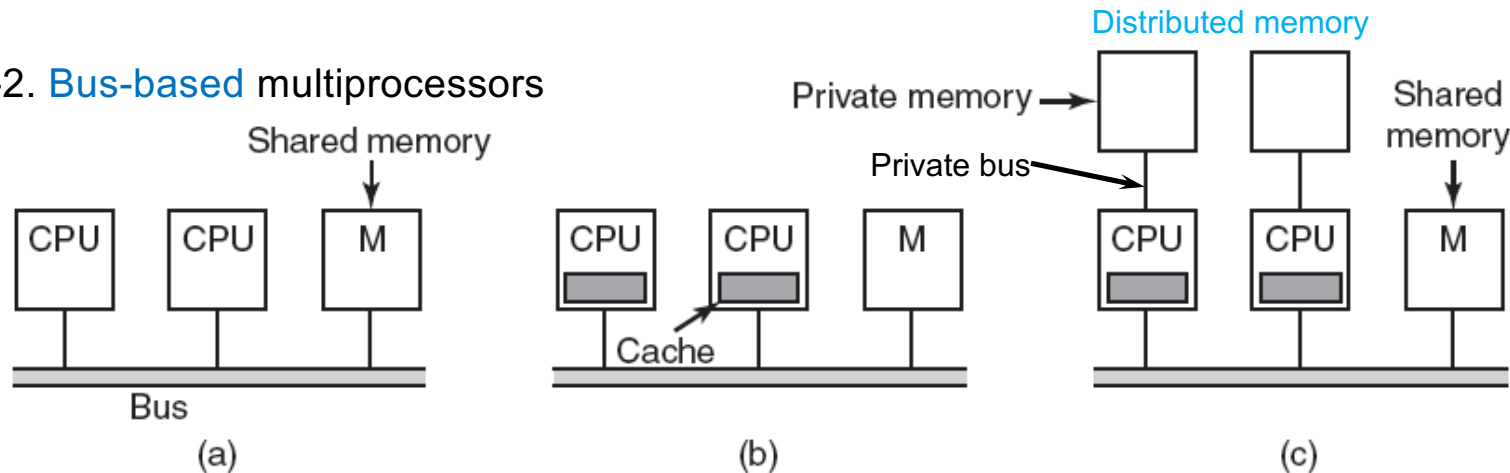
LAN Types

Blocking: network has *fewer transmission paths* than would be required for simultaneous transmissions from all users. $\Rightarrow$ longer communication times as concurrent communication *folded* onto parallel communication system.



LAN > Indirect (Dynamic) > Blocking: UMA Multiprocessors with Bus-Based Architectures

Figure 8-2. Bus-based multiprocessors



Without caching: contention problem, limited scalability (32/64)

Requires arbiter: Hardware solution for atomic memory access for concurrency on shared memory

- ++ Distance between any two nodes: $O(1)$
- ++ Scalable cost $O(p)$, $p = \# \text{CPUs}$
- ++ Ideal for broadcasting
- CPUs idle when many CPUs contend for the bus
- Limited performance scalability, $p=32, 64$

With caching: less bus traffic, increased scalability (>64)

Requires: Cache-coherence protocols (expensive)

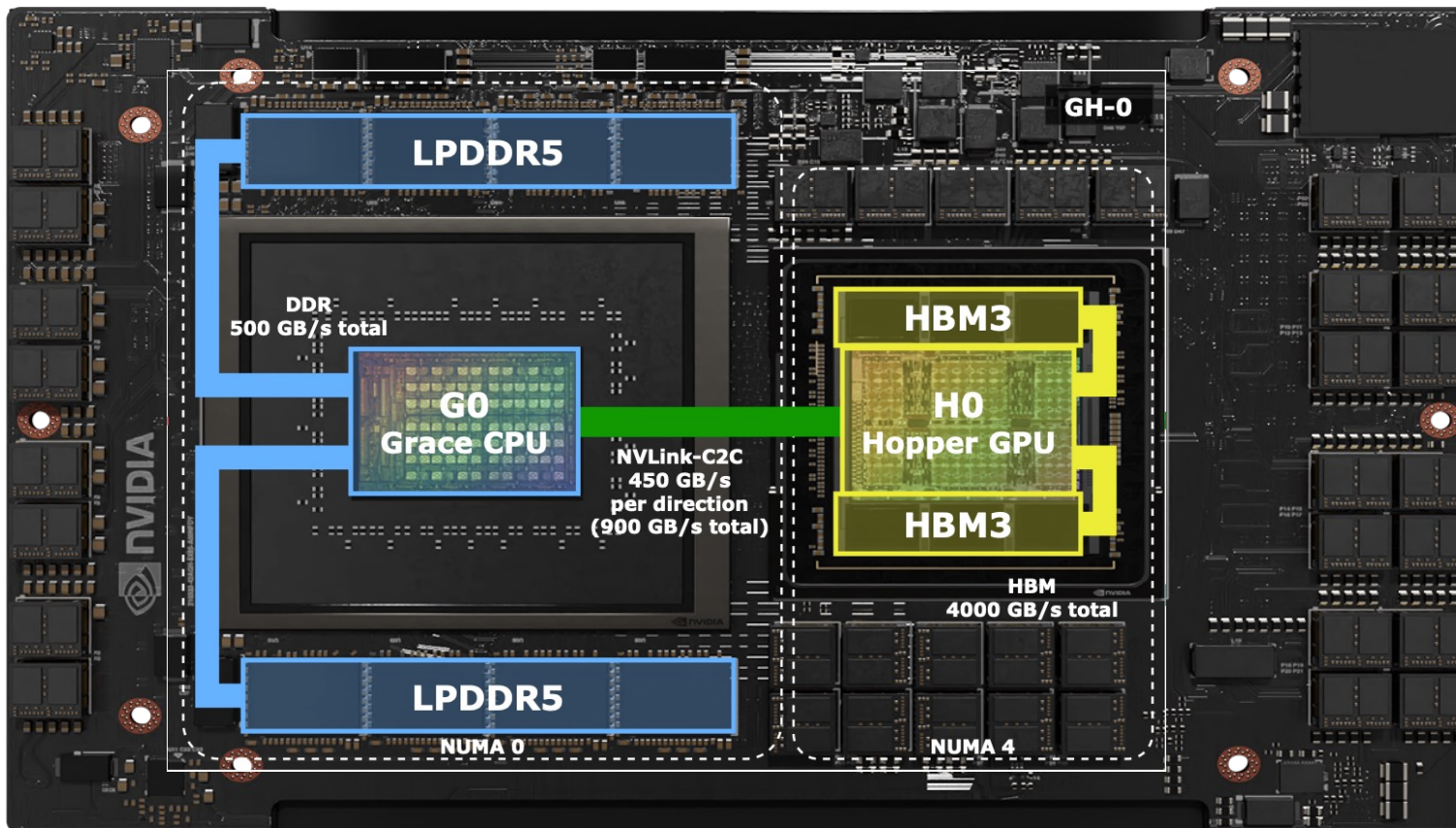
- ++ Cache inside/next to CPU chip, on processor board, or combined $\rightarrow$ fast processing
- ++ Increased scalability $p > 64$
- ++ Cache block: 32/64 B, read-only, read-write
- Cache-coherence protocols needed (cost)

With caching and private memories: less bus traffic, increased scalability (>64)

Requires: Compiler support

- ++ Private memory accessed over private bus
- ++ Compiler places program text / strings / constants / data / stacks / variables in private memories $\rightarrow$ no programmer action
- ++ Decreased bus traffic via careful placement
- Cache-coherence protocols needed now between both private and shared memory (costs more than in (b))

🔗 LAN > Indirect (Dynamic) > Blocking:
UMA Multiprocessors with Bus-Based Architectures
Famous Example: Nvidia's Grace Hopper Superchip



Cache coherent
NVLink for Grace CPU
to Hopper GPU

10 NVLinks total 450
GB/s per direction

C2C: chiplet to chiplet,
alternative to PCI

Present in ALPS

LAN > Indirect (Dynamic) > Blocking: UMA Multiprocessors Using Multistage Interconnection/Switching Network: Basic Elements

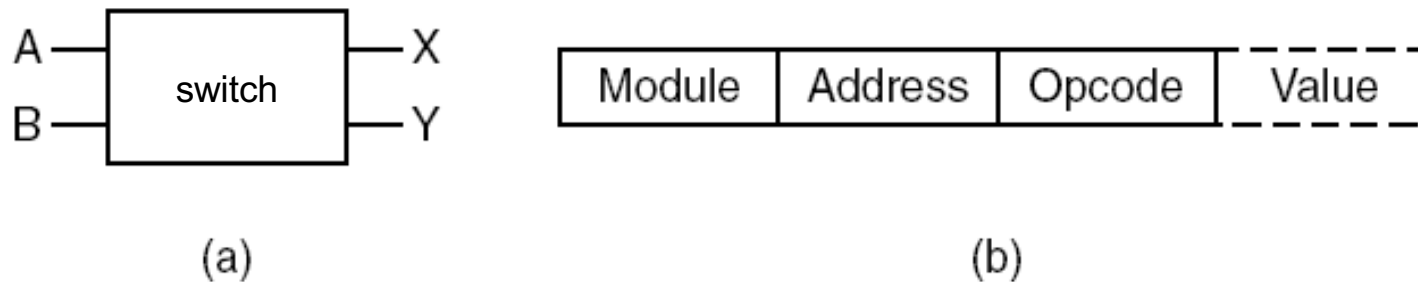


Figure 8-4.

(a) A 2×2 switch with two input lines, A and B, and two output lines, X and Y

(b) A message format, containing the module field (e.g., 011 – see next slides)

- Messages arriving on A or B can be switched to X or Y
Module field/bank tells the switch (fig. a) **which memory module** to use: **X** or **Y**.
Address specifies an **address** within a memory module.
Opcode gives the **operation** to perform at the memory address, such as **READ** or **WRITE**.
Value field (*optional*) may contain an **operand**, such as a 32-bit word to be written on a **WRITE** operation.
- Can be arranged to build larger **multistage** switching networks (e.g., Omega network, next slides).

LAN > Indirect (Dynamic) > Blocking: UMA Multiprocessors Using Multistage Interconnection/Switching Network: General Topology

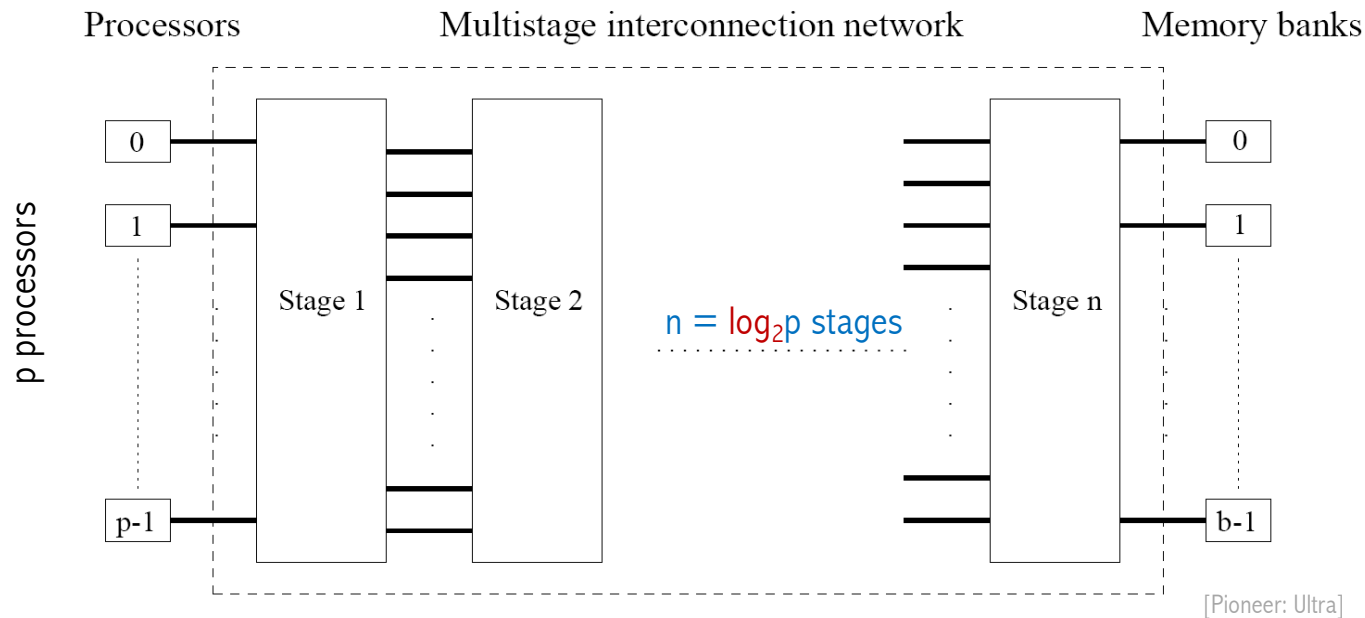


Figure 2.9 The schematic of a typical multistage interconnection network.

++ $\Theta(p \times \log_2 p)$ switching elements $< \Omega(p^2)$ for non-blocking crossbar networks

Solves the scaling cost of crossbars by **splitting the large crossbar switching into several stages of fewer interconnected switches**
 $\Rightarrow$ any destination reachable from any source (via one pass through switch fabric)

- - **Blocking** (conflicts among multiple p over the use of a wire or switch, requests/replies to/from memory)
- - Bandwidth limited

LAN > Indirect (Dynamic) > Blocking: UMA Multiprocessors Using Multistage Interconnection/Switching Network: Omega Example

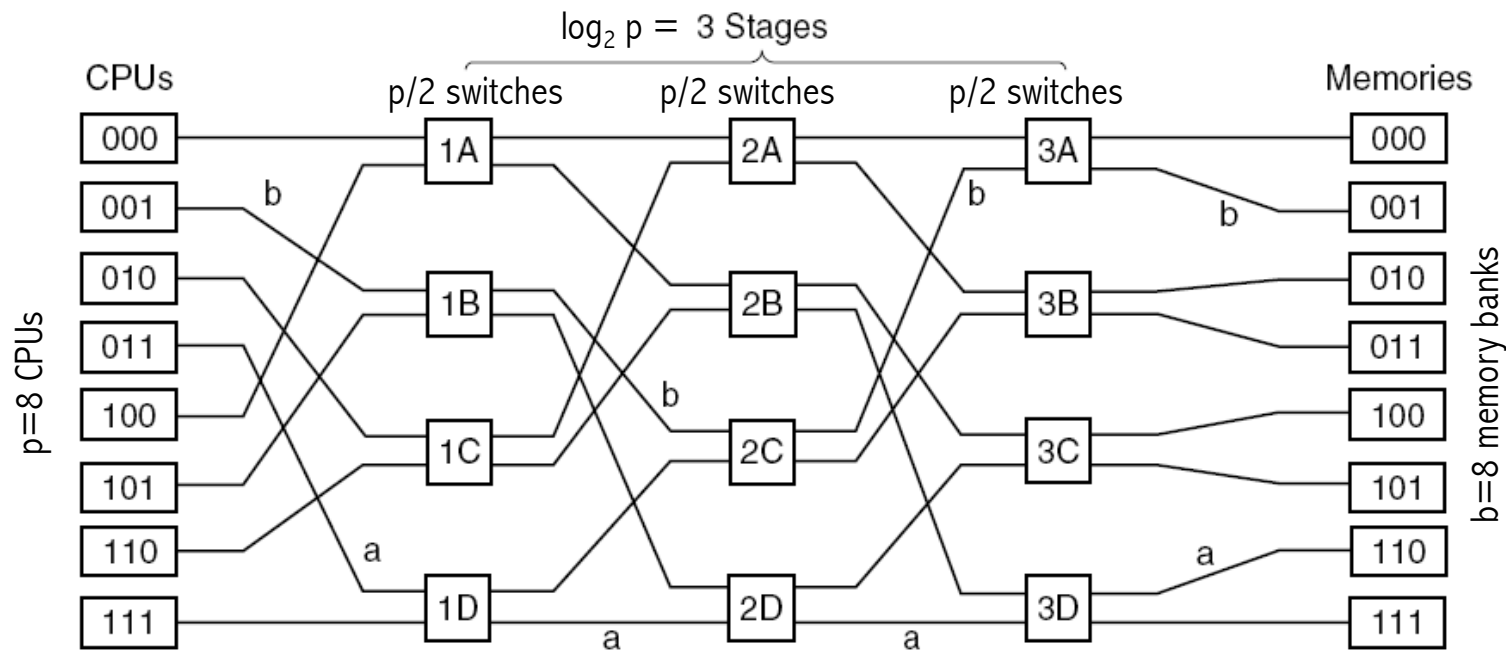


Figure 8-5. An Omega switching network. p CPUs connected to b memory banks using $p/2$ switches in $\log_2 p$ stages.

Perfect-shuffle exchange = interconnection pattern (perfect-shuffle permutation) + exchange switches

LAN > Indirect (Dynamic) > Blocking: UMA Multiprocessors Using Multistage Interconnection/Switching Network: Omega Example

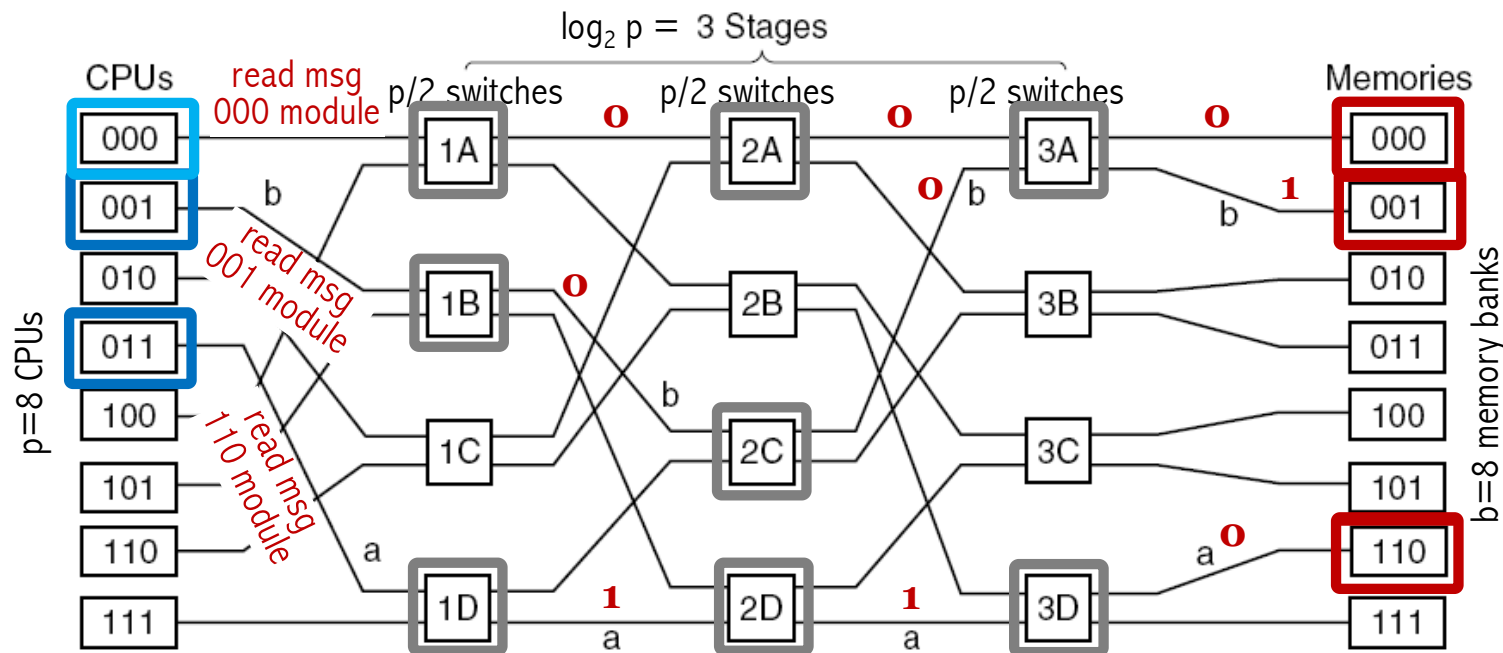


Figure 8-5. An **Omega** switching network. p CPUs connected to b memory banks using $p/2$ switches in $\log_2 p$ stages.

$\Theta(p/2 \times \log_2 p)$ switching elements $< \Theta(p \times \log_2 p)$ for MINs $< \Omega(p^2)$ for non-blocking crossbar networks

Perfect-shuffle exchange = interconnection pattern (perfect-shuffle permutation) + exchange switches

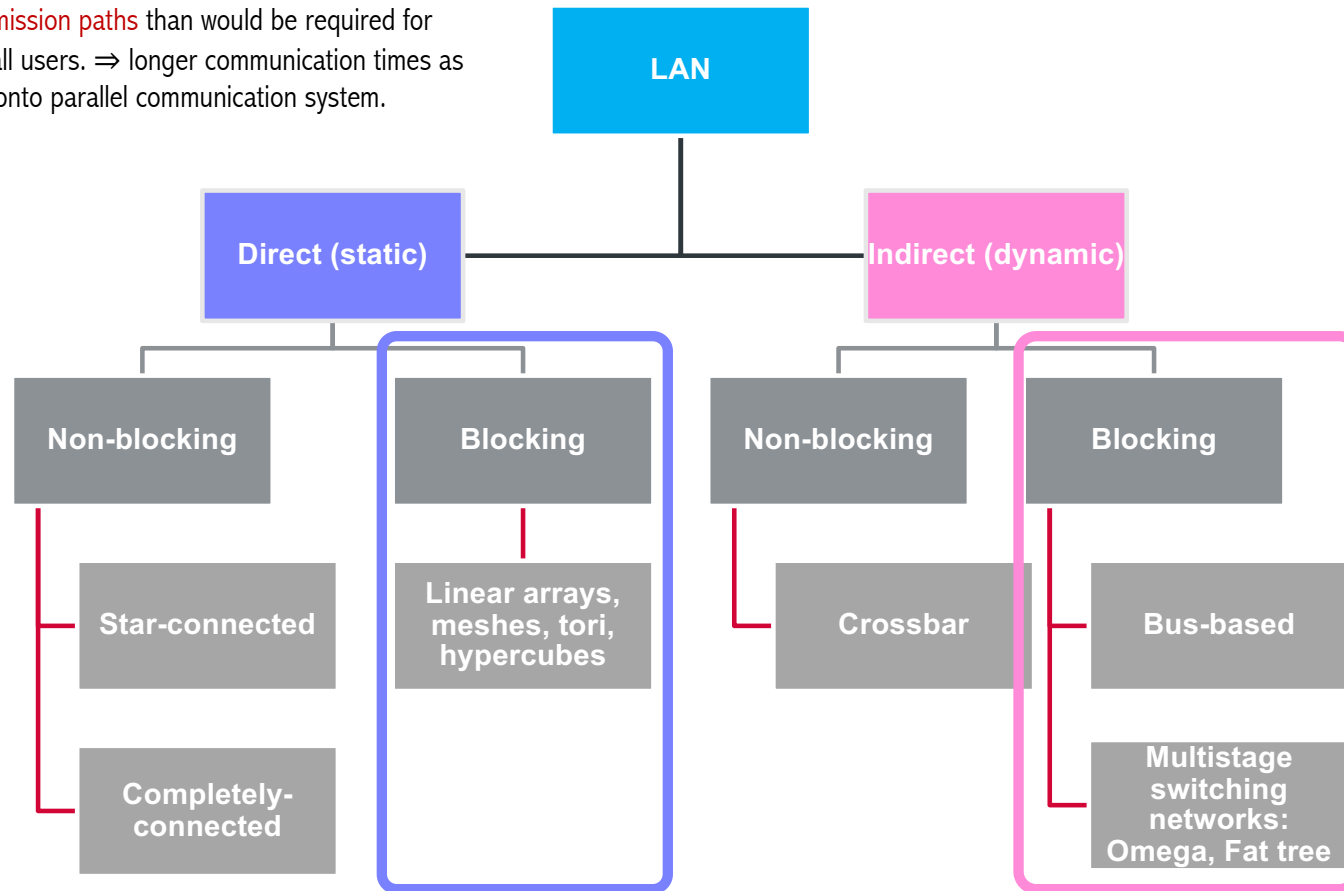
++ **Reduced cost (50%)** in terms of switching elements vs. crossbar networks

-- **Blocking** (conflicts among multiple p over the use of a wire or switch, requests/replies to/from memory)

Fixed Network Type 3: Local Area Networks (LAN)

LAN Types

Blocking: network has *fewer transmission paths* than would be required for simultaneous transmissions from all users. $\Rightarrow$ longer communication times as concurrent communication *folded* onto parallel communication system.



LAN > Direct (Static) / Indirect (Dynamic) > Blocking: Tree Networks



Figure 2.18. Complete binary tree networks: (a) a direct/static tree network

Direct/static trees: One processing element at each node of the tree.

LAN > Direct (Static) / Indirect (Dynamic) > Blocking: Tree Networks

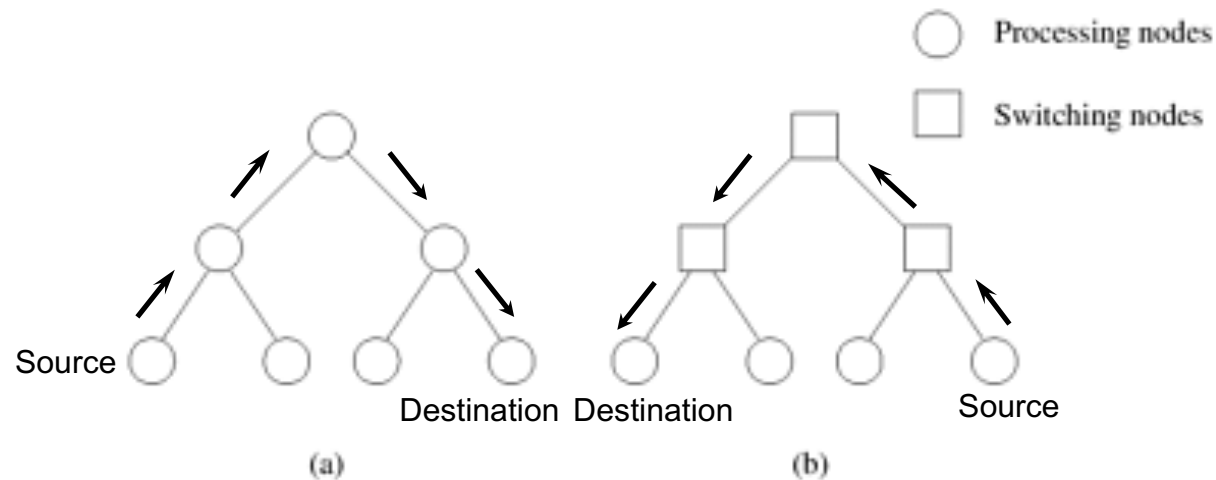


Figure 2.18. Complete binary tree networks: (a) a direct/static tree network; and (b) an indirect/dynamic tree network.

Direct/static trees: One processing element at each node of the tree.

Indirect/dynamic trees: Switching nodes at intermediate tree levels and processing elements at leaf level.

Only one path between any pair of nodes.

To route a message in a tree network

1. The source node sends the message up the tree until it reaches the node at the root of the smallest subtree containing both the source and destination nodes.
2. Then the message is routed down the tree towards the destination node.

- - Tree networks suffer from a communication bottleneck at higher levels of the tree.

LAN > Indirect (Dynamic) > Blocking: Fat Tree Networks

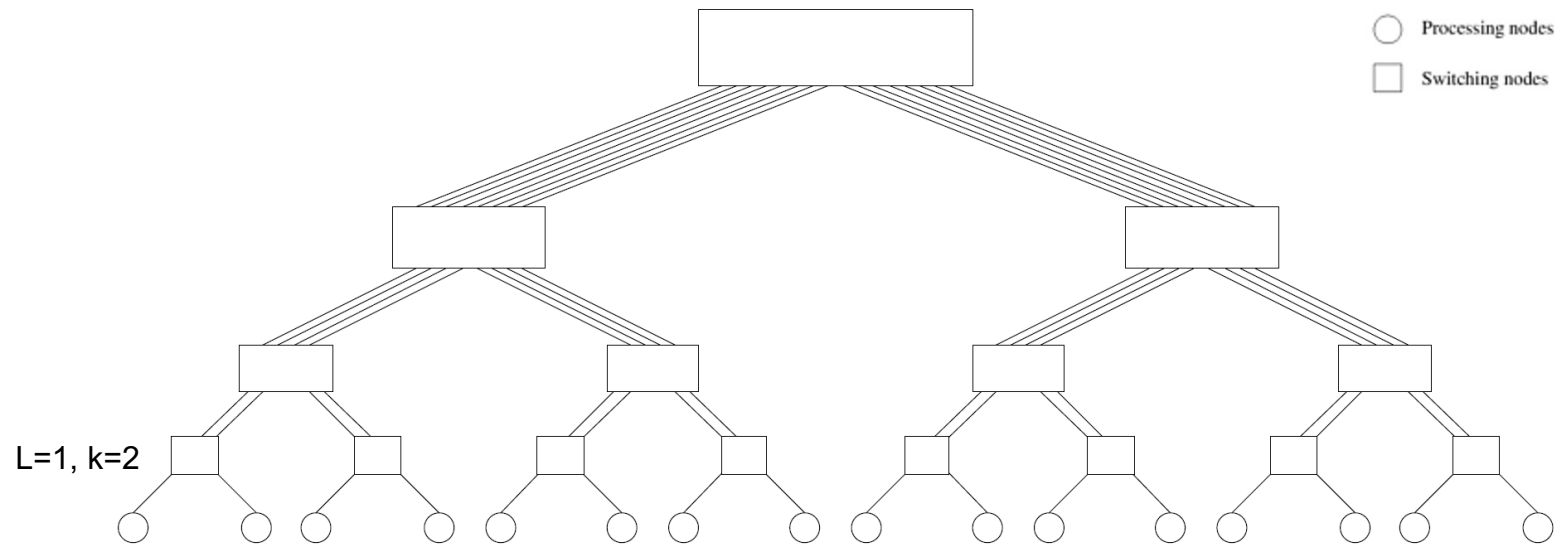


Figure 2.19. A **fat tree** network of $p=16$ processing nodes , $L=4$ levels (1 core, 3 intermediate), and $1+2+4+8=15$ switches  in total

- **Fat tree** = balanced (dynamic) tree network
 - End-node **devices** occupy leaves  of the tree and **switches**  occupy vertices
 - Switches have $2 \times k$ bidirectional/duplex (I/O) ports

++ Alleviate the communication bottleneck at higher levels of dynamic tree networks by **increasing the # of communication links & switching nodes closer to the root** [Leiserson 1985, implemented in TM CM-5].

LAN > Indirect (Dynamic) > Blocking: Fat Tree Networks

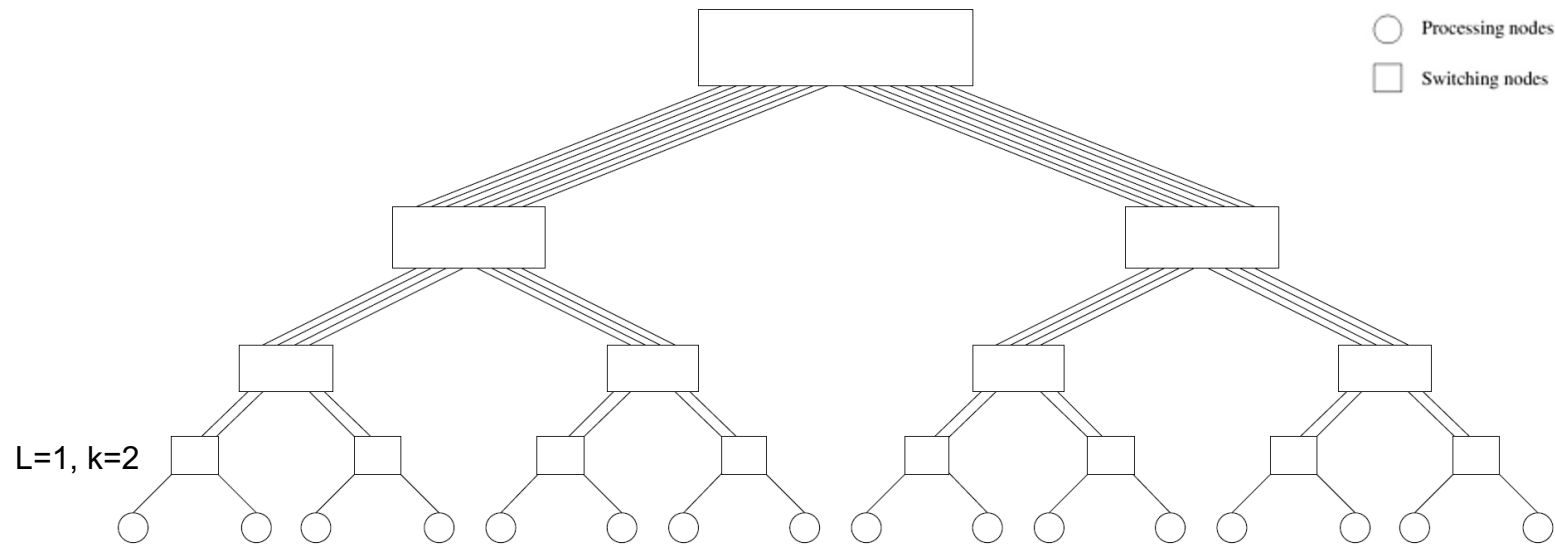


Figure 2.19. A **fat tree** network of $p=16$ processing nodes , $L=4$ levels (1 core, 3 intermediate), and $1+2+4+8=15$ switches  in total

Requires: $(k/2)^{L-1}$ **core switches** (top layer), $2 \times (k/2)^{L-1}$ **intermediate switches**, $p = k^L$ max #nodes.
Also requires sufficient links in each level for constant total link bandwidth across all levels.

++ Recursively **scalable**, partitionable network with simple routing algorithm (up until source/root, down to destination)
++ **Average link distance** between sender/receivers grows only **logarithmically**

- - Can lead to congestion if number of switches/level is smaller

⇒ Most supercomputers use this type of topologies: **miniHPC, Microsoft Azure***



LAN > Static / Dynamic > Blocking and Non-Blocking Evaluation Metrics of Interconnection Networks

Metric	Meaning	Target	Example
Diameter	Maximum number of hops (distance) between any two nodes	↓ smaller	Hypercube < Mesh
Bisection bandwidth	Total data (#messages) that can be transferred in parallel across the network, without messages using the same links/hop	↑ higher	Fat tree < Bus
Arc connectivity	Number of independent paths between nodes (fault tolerance)	↑ higher	Mesh > Ring
Cost	Total # links and # switches needed	↓ lower	Mesh < Crossbar

Trade-offs are needed

- High performance requires high bandwidth and connectivity but comes at high cost
- More affordable networks may deliver less high performance

LAN > Direct (Static) > Blocking and Non-Blocking Evaluation Metrics of Static Interconnection Networks

Table 2.1 A summary of the characteristics of various static network topologies connecting p nodes.

Network	Diameter	Bisection Width	Arc Connectivity	Cost (No. of links)
Completely-connected	1	$p^2/4$	$p - 1$	$p(p - 1)/2$
Star	2	1	1	$p - 1$
Complete binary tree	$2 \log((p + 1)/2)$	1	1	$p - 1$
Linear array	$p - 1$	1	1	$p - 1$
2-D mesh, no wraparound	$2(\sqrt{p} - 1)$	$\sqrt{p}$	2	$2(p - \sqrt{p})$
2-D wraparound mesh	$2\lfloor \sqrt{p}/2 \rfloor$	$2\sqrt{p}$	4	$2p$
Hypercube	$\log p$	$p/2$	$\log p$	$(p \log p)/2$
Wraparound k -ary d -cube	$d\lfloor k/2 \rfloor$	$2k^{d-1}$	$2d$	dp

LAN > Dynamic (Indirect) > Blocking and Non-Blocking Evaluation Metrics of Dynamic Interconnection Networks

Table 2.2 A summary of the characteristics of various dynamic network topologies connecting p processing nodes.

Network	Diameter	Bisection Width	Arc Connectivity	Cost (No. of links)
Crossbar	1	p	1	p^2
Omega Network	$\log p$	$p/2$	2	$p/2$
Dynamic Tree	$2 \log p$	1	2	$p - 1$

Recall: Cost is total number of links and switching nodes (not shown here)

Parallel Computers and Their Programming

What We Learned So Far

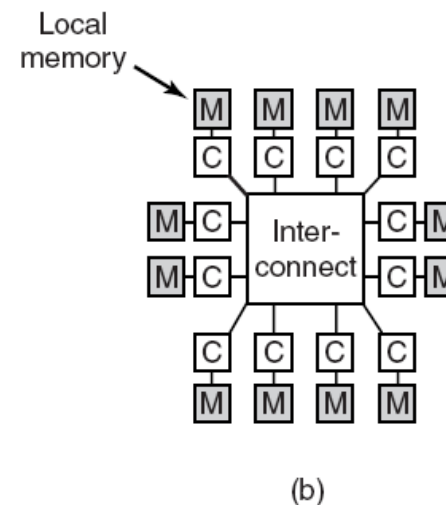
A. Memory architectures

B. Parallel programming models

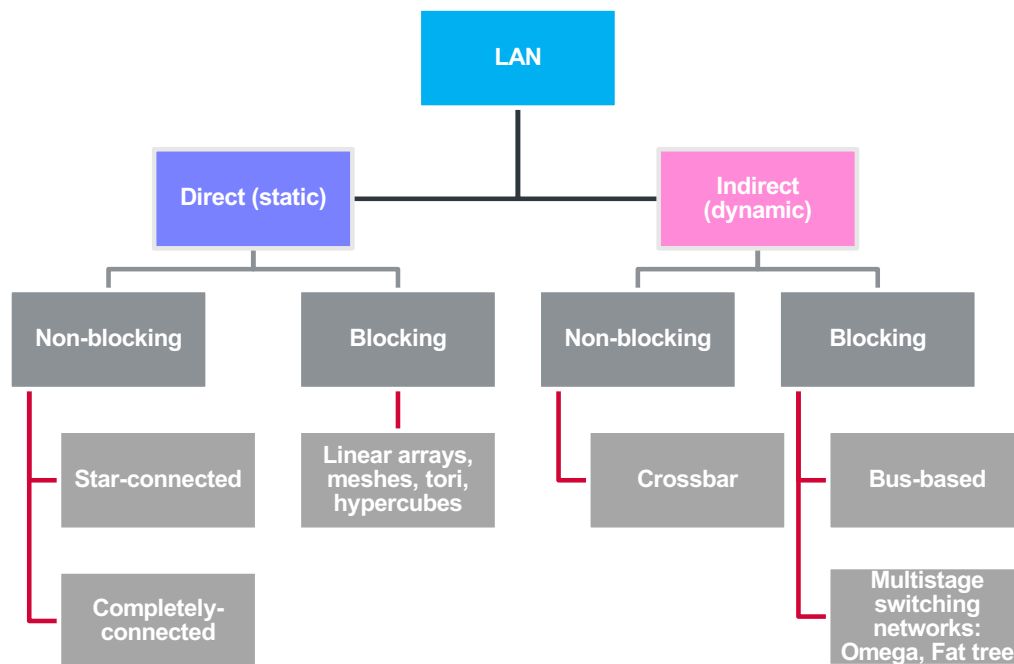
C. Network topologies

1. On chip networks
2. System/Storage area networks
3. Local area networks
4. Wide area networks (optional material)

What the machine offers
w.r.t. communication



What Have We Learned Today?



a Memory Architectures

b Parallel Programming Models

c Network Topologies

Summary

- “The *interconnection network* is the *heart* of parallel architecture.”
[Chuan-Lin and Tse-Yun Feng, Interconnection Networks for Parallel and Distributed Processing (1984)]
- Network topologies are important for all par. steps
- Direct (static) & indirect (dynamic, switched) networks
- Operate in a non- & blocking (folding) fashion
- Examples: D-Torus networks (TPU pods by Google) and Dragonfly topologies (implemented in Slingshot by HPE/Cray) are at the heart of many supercomputers & widely used today

What Have We Learned in This Chapter 5.2?

Parallel Architectures and Programming Models

A. Memory architectures

MM1: Shared memory

MM2: Distributed-memory

MM3: Hybrid distributed shared memory (1+2)

MM4: Distributed shared memory (var. of 3)

MM5: Fully distributed systems

B. Programming models

PM1. Shared memory (without threads)

PM2. Shared memory (with threads)

PM3. Distributed memory / Message Passing

PM4. Data parallel

PM5. Hybrid distributed shared memory

PM6. Single Program Multiple Data (SPMD)

PM7. Multiple Program Multiple Data (MPMD)

C. Network topologies

T1. On chip networks

T2. System/Storage area networks

T3. Local area networks

T4. Wide area networks

a Memory Architectures

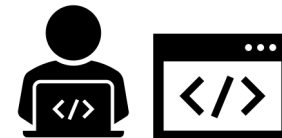
b Parallel Programming Models

c Network Topologies

Summary

- **Parallel** Computing: A **Memory** ↔ B **Programming** ↔ C **Networks**
A: 4+1 memory models, B: 5+2 programming models, C: 4 network types
- **Parallel** programs = **parallel** algorithms + **parallel** data structuring
(+ **parallel** hardware view*)
* A: memory architectures and B: network topologies

... Now, let's get on to Step 4: **parallel** programming in Ch 5.3 😊





University
of Basel

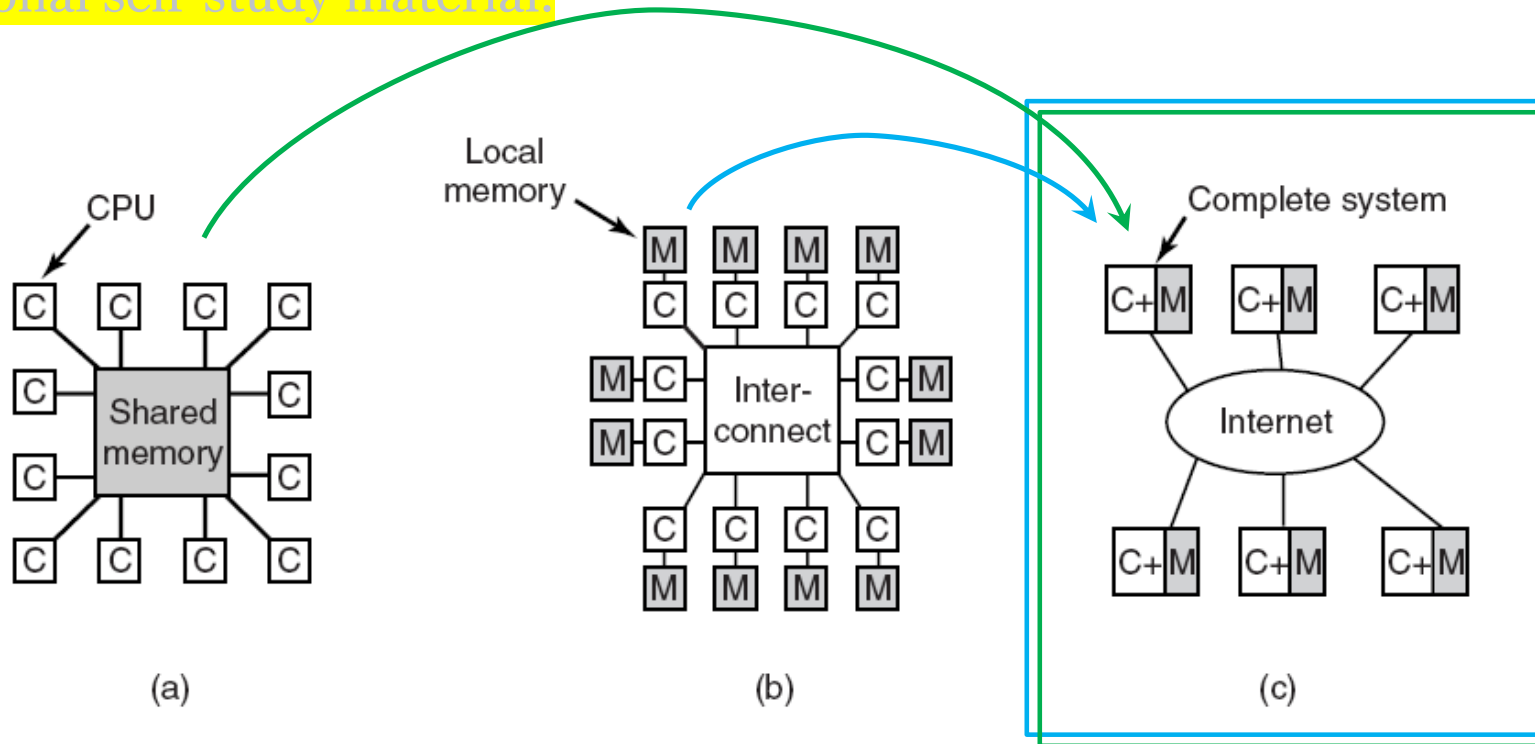
Wide-Area Distributed Systems

Start: Additional Material

Note: Following slides are offered as optional self-study material. NOT for the exam!

Multiple Processor Computer Systems

Note: Optional self-study material.



(a) A shared-memory multiprocessor

(b) A parallel distributed-memory (message-passing) multicomputer

(c) A wide area distributed multicomputer system with shared- (locally) and/or distributed-memory (globally) units

Distributed Computing

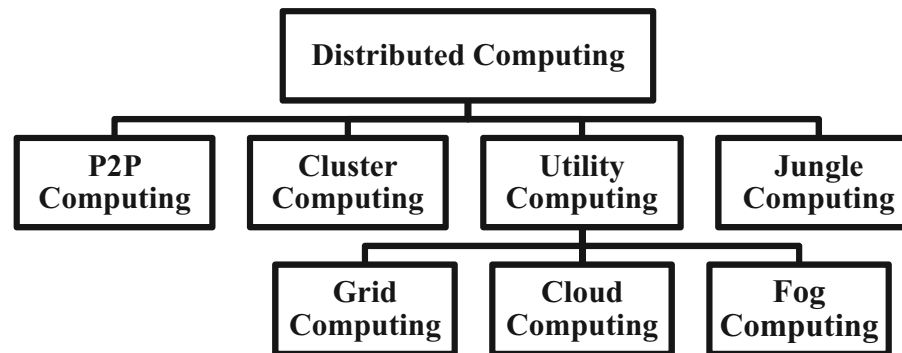


Figure 1. Taxonomy of distributed computing.

[Source: A review on modern distributed computing paradigms: Cloud computing, jungle computing and fog computing, 2014, DOI: 10.2498/cit.1002381]

Network Type 4: Wide Area Networks

Distributed Computing → P2P Computing

Each node

- Client and/or a server

Peer machines

- Client computers connected to the Internet.

Client machines

- Autonomously & freely join/leave the system

No coordinator-worker relationship exists.

No central coordination.

No central database is needed.

⇒ No global system view by any peer machine

⇒ Self-organizing system with distributed control

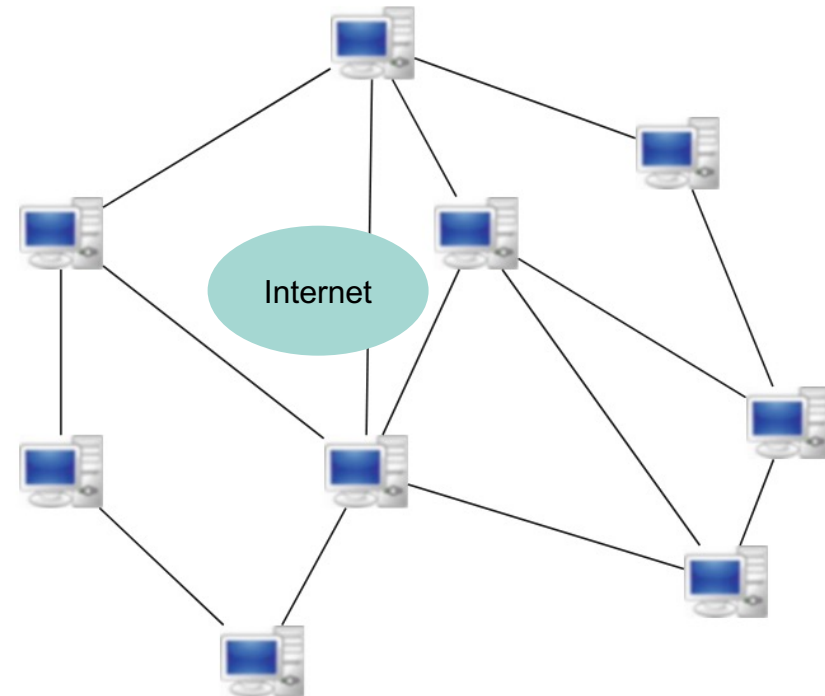


Fig. 9.2 Unstructured P2P network

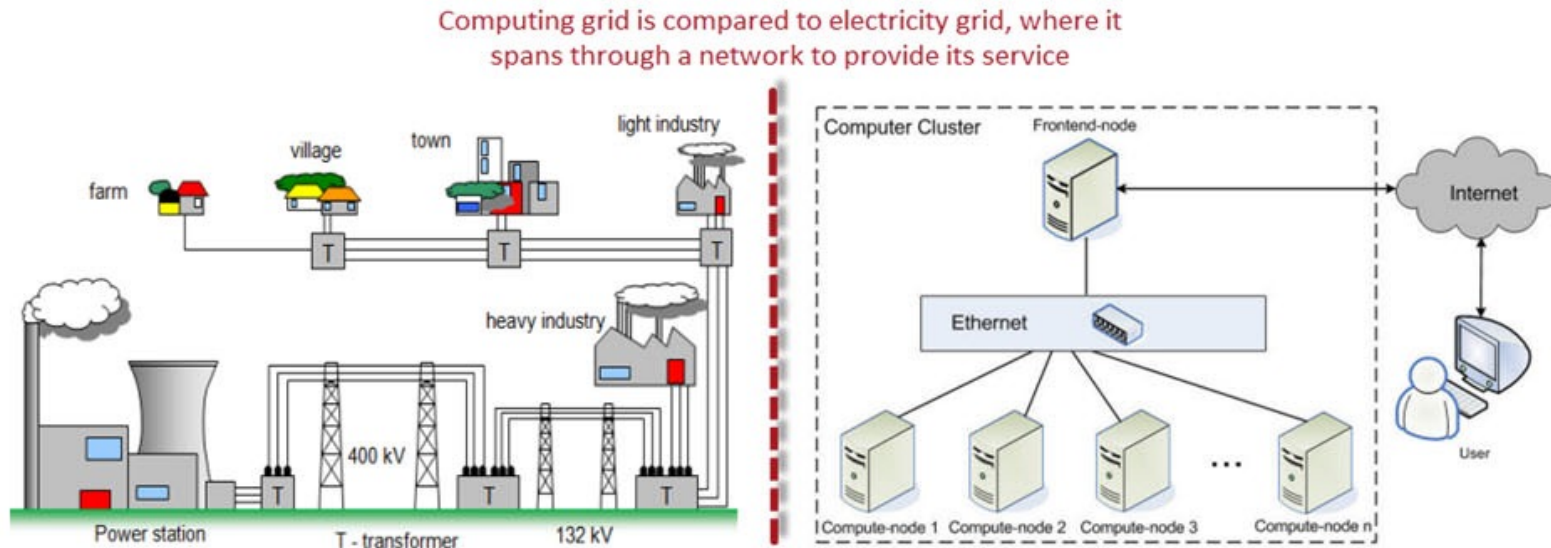
M. Tamer Özsu and Patrick Valduriez, Principles of Distributed Database Systems (Springer International Publishing, 2020), doi:[10.1007/978-3-030-26253-2](https://doi.org/10.1007/978-3-030-26253-2).

[Source: <https://www.guru99.com/cloud-computing-for-beginners.html>]

[Jungle Computing 2011 Book Chapter: <http://rvannieuwpoort.synology.me/papers/gcv-book-chapter.pdf>]

Network Type 4: Wide Area Networks

Utility Computing → Grid Computing



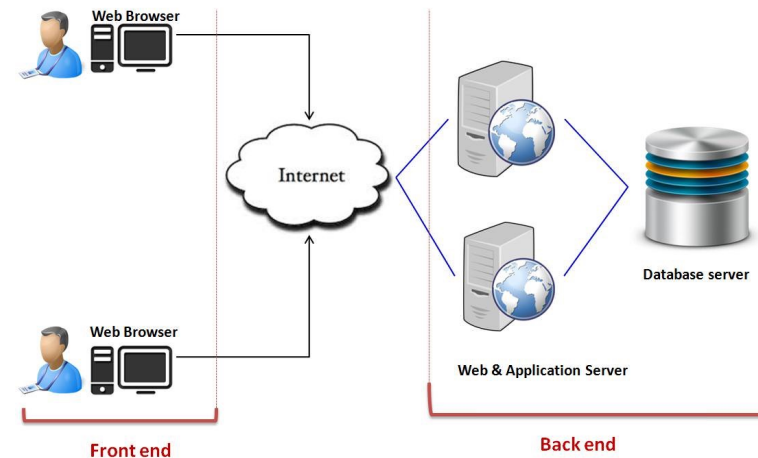
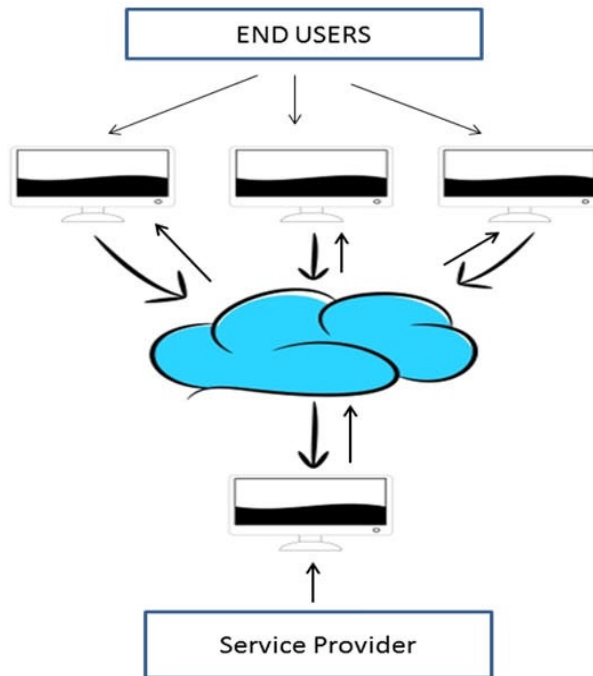
- ++ High performance
- ++ Distributed
- ++ Secure
- ++ Fault-tolerance
- ++ Transparent

'Grid Computing' is an infrastructure that links computing resources such as PCs, servers, workstations and storage elements and provides the mechanism required to access them.

[Source: <https://www.quru99.com/cloud-computing-for-beginners.html>]

Network Type 4: Wide Area Networks

Utility Computing → **Cloud** Computing



The term “**Cloud**” came from a network design that was used by network engineers to represent the *location of various network devices* and their interconnection. The shape of this network design was like a cloud.

[Source: <https://www.guru99.com/cloud-computing-for-beginners.html>]

Network Type 4: Wide Area Networks

Utility Computing → Cloud Computing

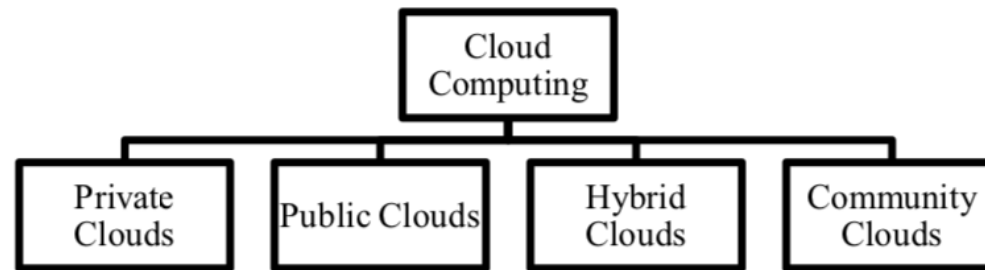


Figure 5.2: Classification of Cloud Computing

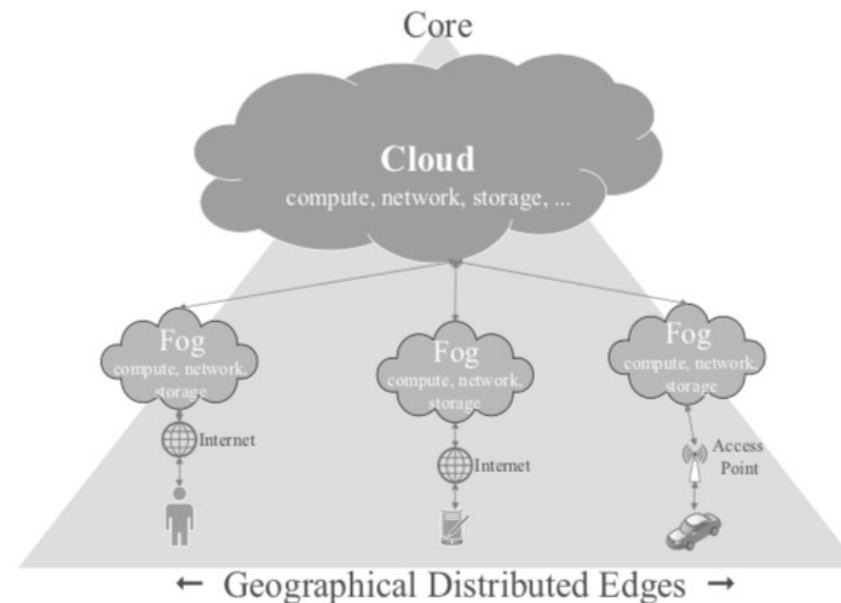
Involves *outsourcing* of *computing resources* with the capabilities of *expendable resource scalability*, *on-demand provisioning* with little or no up-front IT infrastructure investment costs.

Types of service (delivery models)

- *Infrastructure*-as-a-service (IaaS)
- *Platform*-as-a-service (PaaS)
- *Software*-as-a-Service (SaaS)

[Source: <https://arxiv.org/pdf/1311.3070.pdf>]

Network Type 4: Wide Area Networks: Utility Computing → Fog Computing



“A systematic, highly virtual, secure, and network-integrated platform that provides **computing**, **storage**, and **networking services** between end points (users) and traditional Cloud computing data centers” *

A model in which data, processing, and applications are

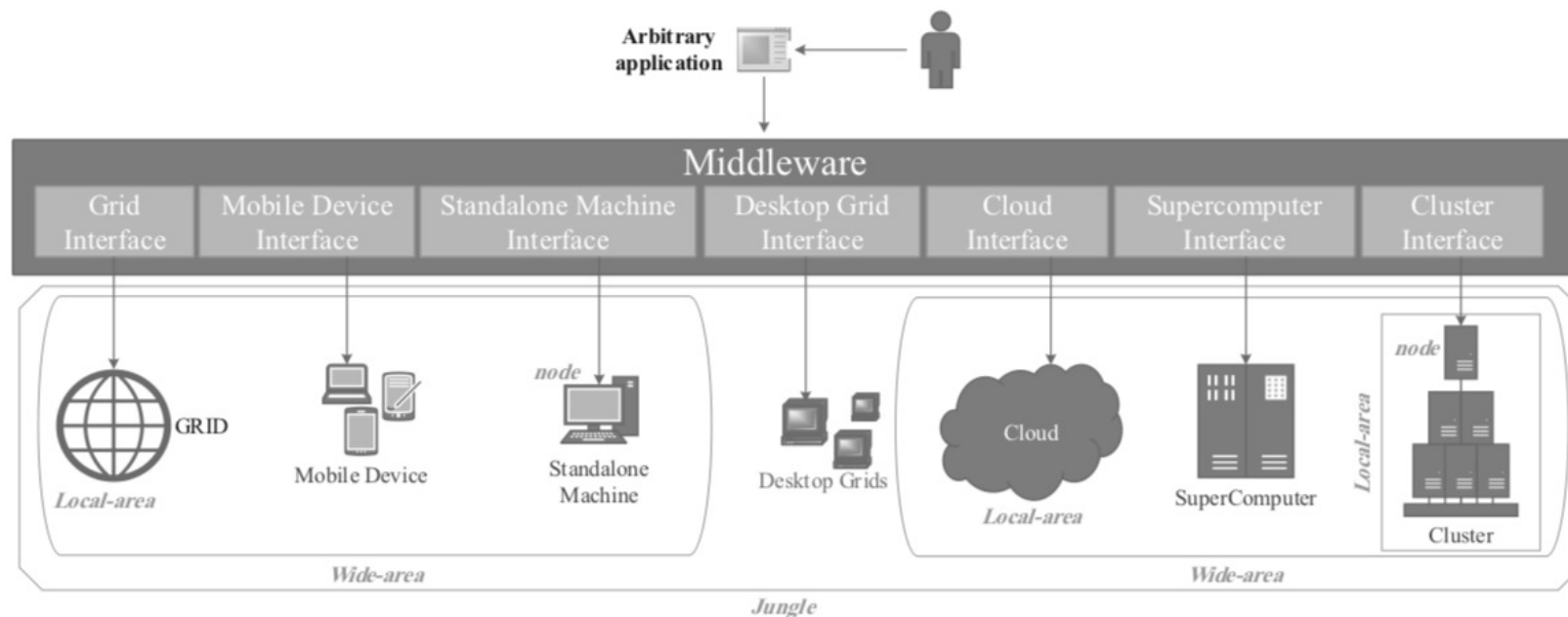
- Concentrated *in devices* at the *network edge*, rather than existing almost entirely in the Cloud.
- Isolated from the Cloud and placed closer to the end-user

* [Source: Bonomi, Milito, Zhu, & Addepalli, 2012]

[Source: A review on modern distributed computing paradigms: Cloud computing, jungle computing and fog computing, 2014, DOI: 10.2498/cit.1002381]

Network Type 4: Wide Area Networks

Jungle Computing



“... *simultaneous use of multiple platforms* (e.g., clusters, grids, and clouds used concurrently).

A true *computing jungle*.” *

A form of highly *heterogeneous distributed computing*.

* [Jungle Computing 2011 Book Chapter: <http://rvannieuwpoort.synology.me/papers/gcv-book-chapter.pdf>]

[Source: A review on modern distributed computing paradigms: Cloud computing, jungle computing and fog computing, 2014, DOI: 10.2498/cit.1002381]

Network Type 4

Grid Vs. Utility Vs. Cloud Computing

Cloud Computing	Grid Computing
•Cloud computing works more as a service provider for utilizing computer resource	•Grid computing uses the available resource and interconnected computer systems to accomplish a common goal
•Cloud computing is a centralized model	•Grid computing is a decentralized model, where the computation could occur over many administrative model
•Cloud is a collection of computers usually owned by a single party. •	•A grid is a collection of computers which is owned by a multiple parties in multiple locations and connected together so that users can share the combined power of resources
•Cloud offers more services all most all the services like web hosting, DB (Data Base) support and much more	•Grid provides limited services
•Cloud computing is typically provided within a single organization (eg : Amazon)	•Grid computing federates the resources located within different organization.

Utility Computing	Cloud Computing
•Utility computing refers to the ability to charge the offered services, and charge customers for exact usage	•Cloud Computing also works like utility computing, you pay only for what you use but Cloud Computing might be cheaper, as such, Cloud based app can be up and running in days or weeks.
•Utility computing users want to be in control of the geographical location of the infrastructure	•In cloud computing, provider is in complete control of cloud computing services and infrastructure
•Utility computing is more favorable when performance and selection infrastructure is critical	•Cloud computing is great and easy to use when the selection infrastructure and performance is not critical
•Utility computing is a good choice for less resource demanding	•Cloud computing is a good choice for high resource demanding
•Utility computing refers to a business model	•Cloud computing refers to the underlying IT architecture

[Source: <https://www.guru99.com/cloud-computing-for-beginners.html>]



University
of Basel

Wide-Area Distributed Systems

End: Additional Material

Note: Earlier slides were offered as optional self-study material. NOT for the exam!